

## СОДЕРЖАНИЕ

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>ВВЕДЕНИЕ .....</b>                                                          | <b>3</b>  |
| <b>ГЛАВА 1. СИСТЕМА ПАРСИНГА ДАННЫХ С АУКЦИОНА STALCRAFT .....</b>             | <b>4</b>  |
| 1.1. Архитектура и общая схема работы парсера.....                             | 4         |
| 1.2. Подсистема поиска по названию .....                                       | 6         |
| 1.3. Иерархическая система поиска по категориям.....                           | 10        |
| 1.4. Детальный парсинг информации о предметах.....                             | 14        |
| 1.5. Обработка ошибок и обеспечение надежности.....                            | 15        |
| 1.6. Производительность и оптимизация .....                                    | 15        |
| <b>ГЛАВА 2. ИНТЕГРАЦИЯ С ОФИЦИАЛЬНЫМ API STALCRAFT</b>                         | <b>17</b> |
| 2.1. Архитектура и общая схема работы API клиента .....                        | 17        |
| 2.2. Механизм аутентификации OAuth 2.0 .....                                   | 18        |
| 2.3. Формирование HTTP-запросов и обработка ответов .....                      | 21        |
| 2.4. Получение данных об аукционных лотах.....                                 | 23        |
| 2.5. Обработка пагинации .....                                                 | 25        |
| 2.6. Дополнительные методы работы с данными .....                              | 26        |
| 2.7. Обработка ошибок и обеспечение надежности.....                            | 27        |
| 2.8. Производительность и оптимизация .....                                    | 28        |
| 2.9. Заключение по главе .....                                                 | 30        |
| <b>ГЛАВА 3. ТЕЛЕГРАММ-БОТ ДЛЯ ПОИСКА ПРЕДМЕТОВ НА АУКЦИОНЕ STALCRAFT .....</b> | <b>31</b> |
| 3.1. Архитектура и общая схема работы бота .....                               | 31        |
| 3.2. Модули бота и их взаимодействие .....                                     | 32        |
| 3.3. Система авторизации и безопасности .....                                  | 34        |
| 3.4. Пользовательский интерфейс и навигация .....                              | 40        |
| 3.5. Поиск предметов и вывод информации.....                                   | 43        |
| 3.6. Административные функции.....                                             | 48        |
| 3.7. Обработка ошибок и логирование .....                                      | 59        |
| 3.8. Производительность и оптимизация .....                                    | 61        |
| 3.9. Заключение по главе .....                                                 | 61        |

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>ГЛАВА 4. ХРАНЕНИЕ ДАННЫХ.....</b>            | <b>64</b> |
| <b>4.1. Общая концепция .....</b>               | <b>64</b> |
| <b>4.2. Данные предметов (Item) .....</b>       | <b>67</b> |
| <b>4.3. Пользователи (User) .....</b>           | <b>69</b> |
| <b>4.4. Модель журнала событий (Log).....</b>   | <b>70</b> |
| <b>4.5. Менеджер баз данных DBManager .....</b> | <b>71</b> |
| <b>4.6. Заключение по главе .....</b>           | <b>71</b> |
| <b>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....</b>    | <b>73</b> |

## ВВЕДЕНИЕ

Мессенджер Telegram, являясь одной из самых популярных платформ для обмена сообщениями, предоставляет широкие возможности для создания ботов - автоматизированных программ, способных взаимодействовать с пользователями через текстовый интерфейс. Разработка телеграм-бота для поиска и анализа аукционной информации позволяет игрокам получать необходимые данные в любом месте и в любое время, используя привычный интерфейс мессенджера. Игровая экономика Stalcraft была выбрана в качестве предметной области данного исследования не по причине специфического интереса к данной игре, а исключительно как понятный и наглядный пример для демонстрации технических возможностей интеграции телеграм-бота с внешними программными интерфейсами (API). Архитектурные решения, методы получения данных, алгоритмы обработки информации и подходы к обеспечению отказоустойчивости, реализованные в данной работе, являются универсальными и могут быть применены для создания аналогичных систем в самых различных предметных областях: от мониторинга цен на товары в интернет-магазинах до отслеживания биржевых котировок и анализа данных.

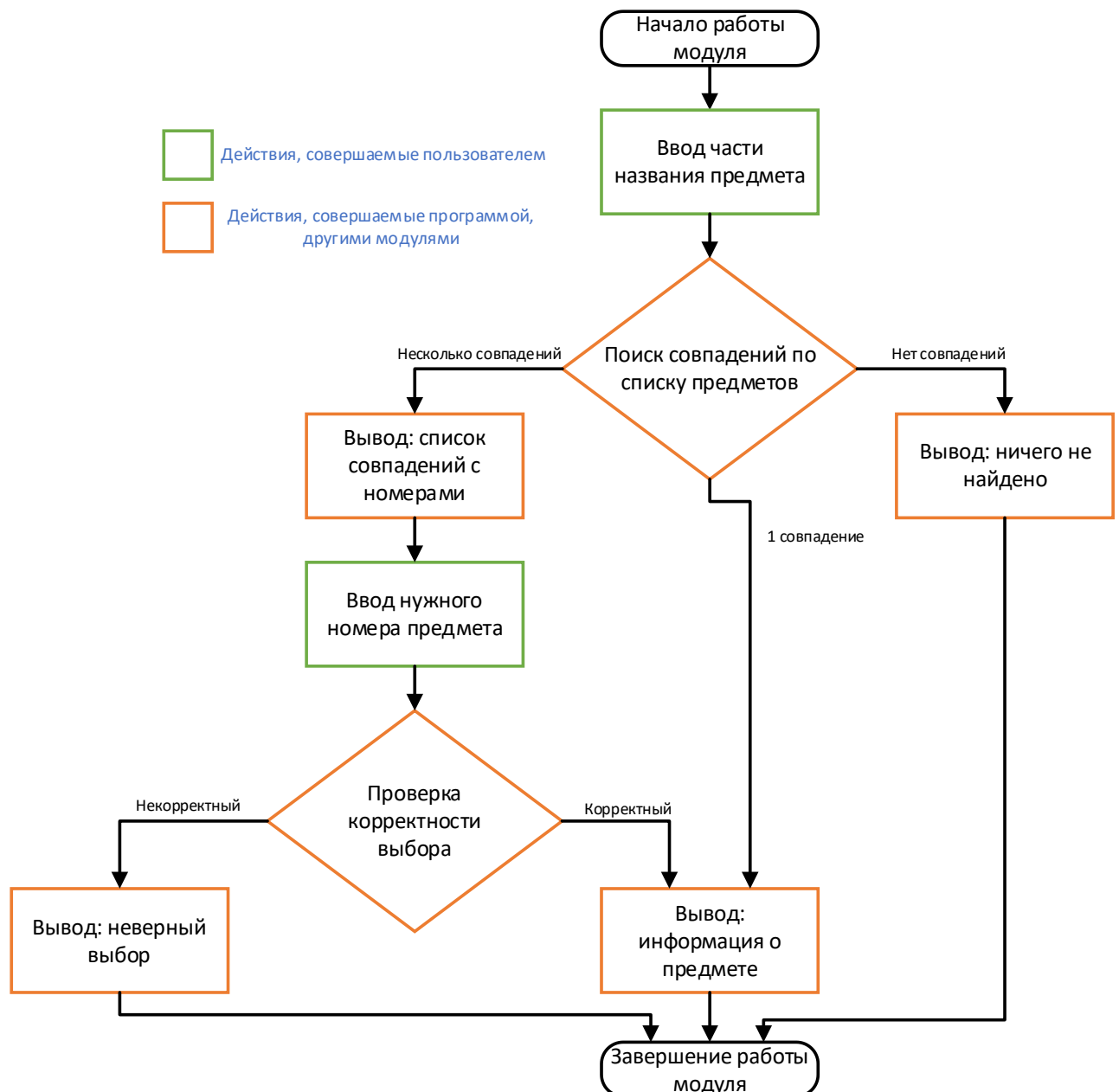
Ручной мониторинг аукциона представляет собой трудоемкий процесс, требующий значительных временных затрат и не гарантирующий получения полной картины рыночной ситуации. В связи с этим возникает потребность в автоматизированных инструментах, способных агрегировать информацию из различных источников, предоставлять ее в удобном для анализа виде и обеспечивать быстрый поиск интересующих предметов.

Цель курсовой работы - разработать телеграм-бота для поиска и анализа аукционной информации игры Stalcraft, обеспечивающего интеграцию с официальным программным интерфейсом игры, резервный механизм получения данных через парсинг веб-интерфейса, удобное пользовательское взаимодействие и надежное хранение информации.

# ГЛАВА 1. СИСТЕМА ПАРСИНГА ДАННЫХ С АУКЦИОНА STALCRAFT

## 1.1. Архитектура и общая схема работы парсера

Система парсинга представляет собой многоуровневую архитектуру, состоящую из нескольких взаимосвязанных модулей, графическое изображение которой представлено на (Блок-схема 1).

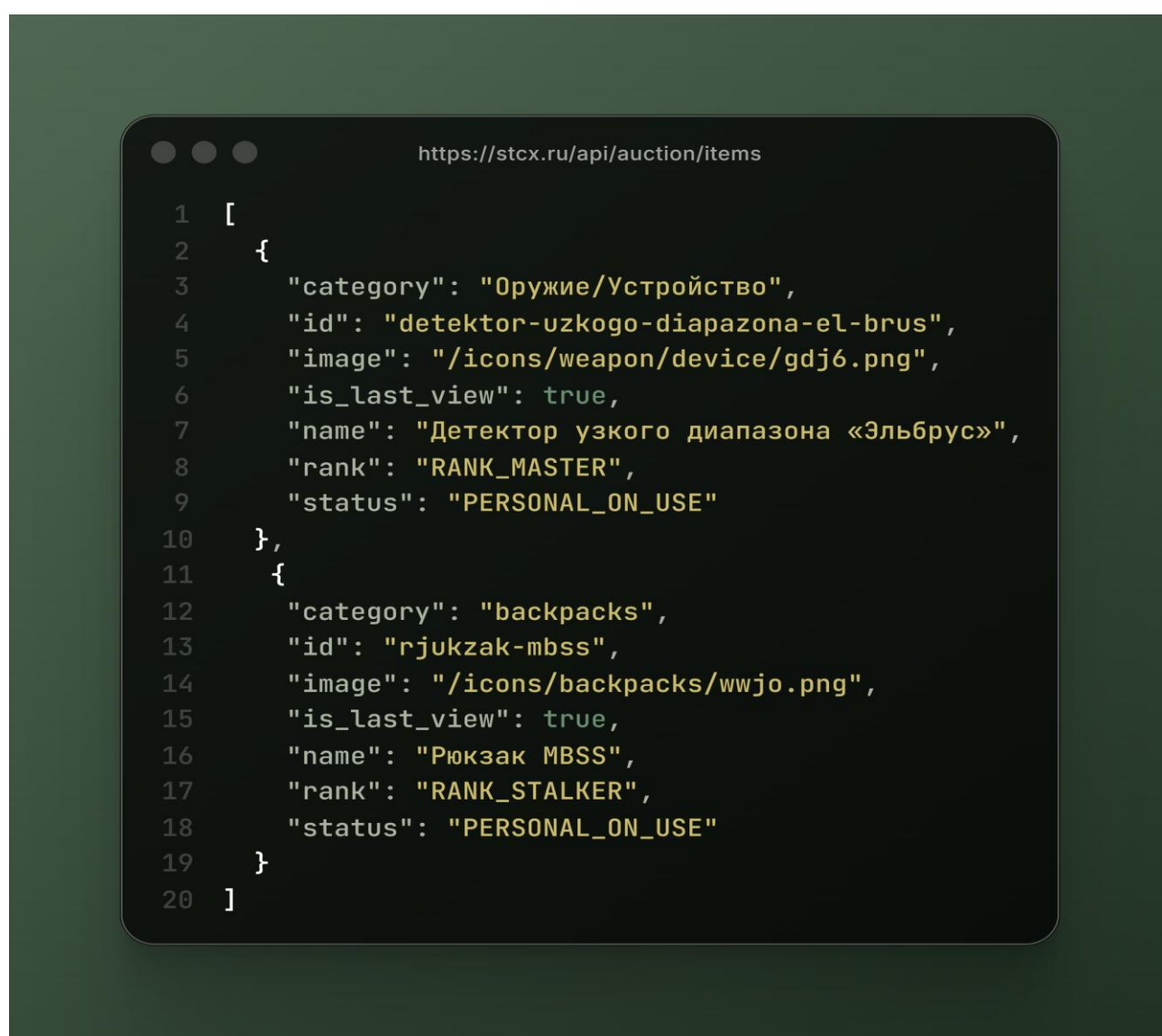


Блок-схема 1. Общая система работы парсера данных

Основной процесс начинается с получения актуальных данных через API аукциона STALCRAFT, после чего следует их обработка, хранение и предоставление пользователю через различные интерфейсы поиска.

### 1.1.1. Инициализация и получение данных

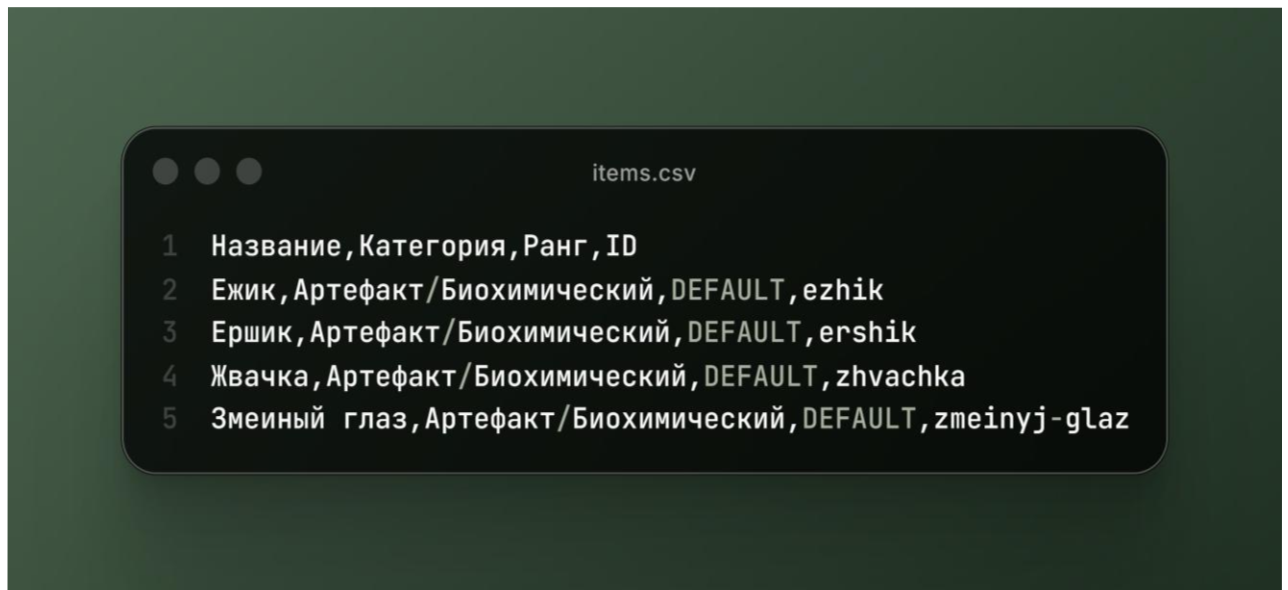
Первым этапом работы системы является загрузка актуального каталога предметов с официального API аукциона. Для этого используется HTTP-запрос к эндпоинту <https://stcx.ru/api/auction/items>. Сервер API возвращает данные в формате JSON, содержащие полный перечень предметов, доступных на аукционе. Вид возвращаемых данных представлен на (Изобр. 1).



Изобр. 1. Вид возвращаемого JSON-ответа с API

### 1.1.2. Обработка и структурирование данных

Полученные данные проходят многоэтапную обработку. На этапе сохранения в CSV-файл происходит критически важная операция локализации категорий предметов. Например, системная категория "backpacks" преобразуется в пользовательское понятие "Рюкзаки". Этот процесс обеспечивает единообразие данных и удобство для русскоязычных пользователей. Каждый предмет сохраняется с четырьмя ключевыми атрибутами: название, категория, ранг и уникальный идентификатор (Изобр. 2).



*Изобр. 2. Вид CSV-файла с сохраненными данными о предметах*

### 1.1.3. Организация хранения данных

Для хранения информации используется CSV-формат, что обеспечивает простоту обработки и человеко-читаемый вид данных. Файл items.csv структурируется со следующими полями:

"Название" - полное наименование предмета

"Категория" - локализованное название категории

"Ранг" - уровень редкости предмета

"ID" - уникальный идентификатор для обращения к API

Важной особенностью является автоматическая сортировка данных по категориям и названиям, что значительно ускоряет последующие операции поиска.

## 1.2. Подсистема поиска по названию

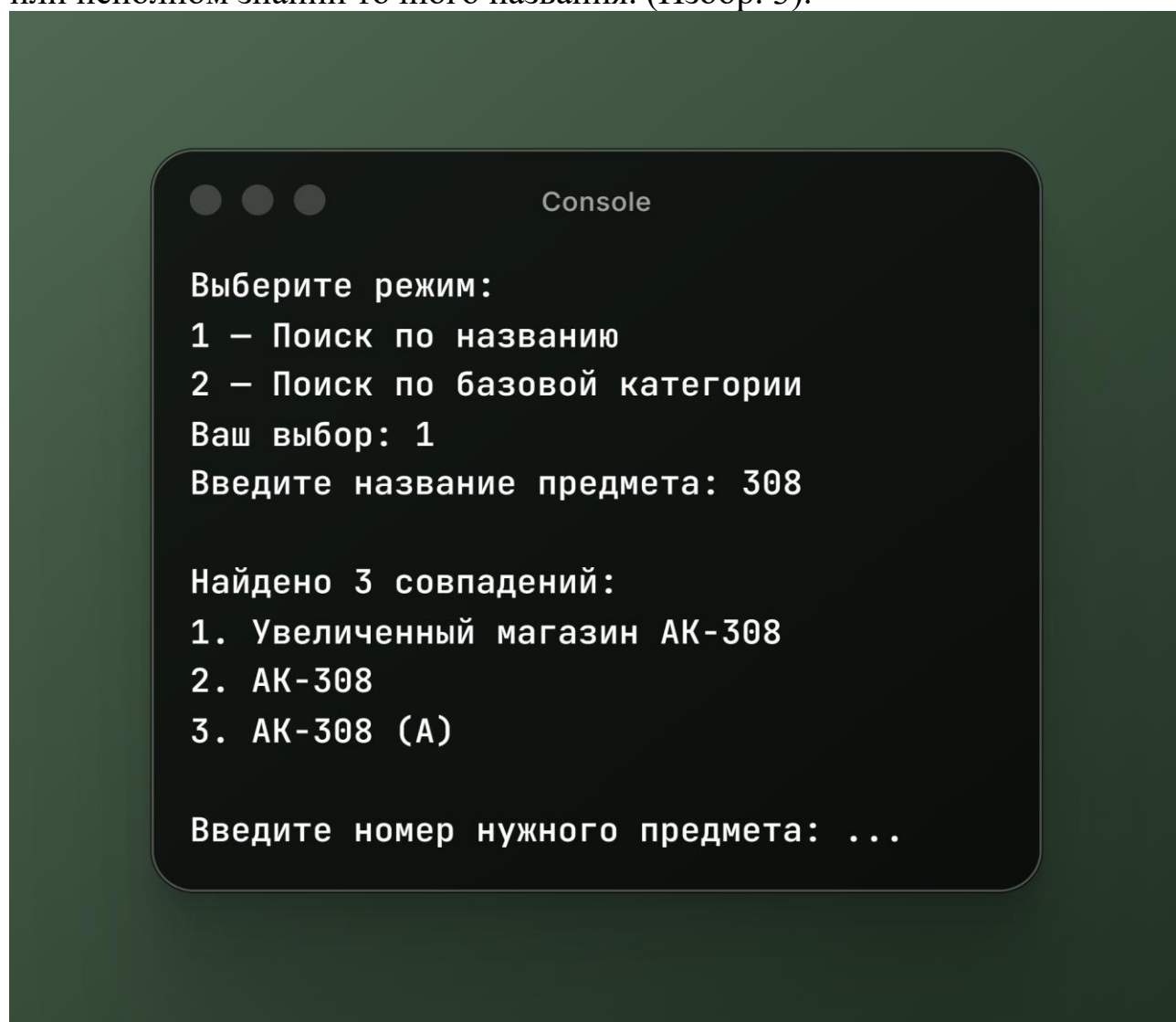
Поиск предмета по названию представляет собой функцию, позволяющую пользователю производить поиск необходимого предмета введя часть

вплоть до полного названия этого предмета. Графическое изображение функции представлено на (Блок-схема 2).

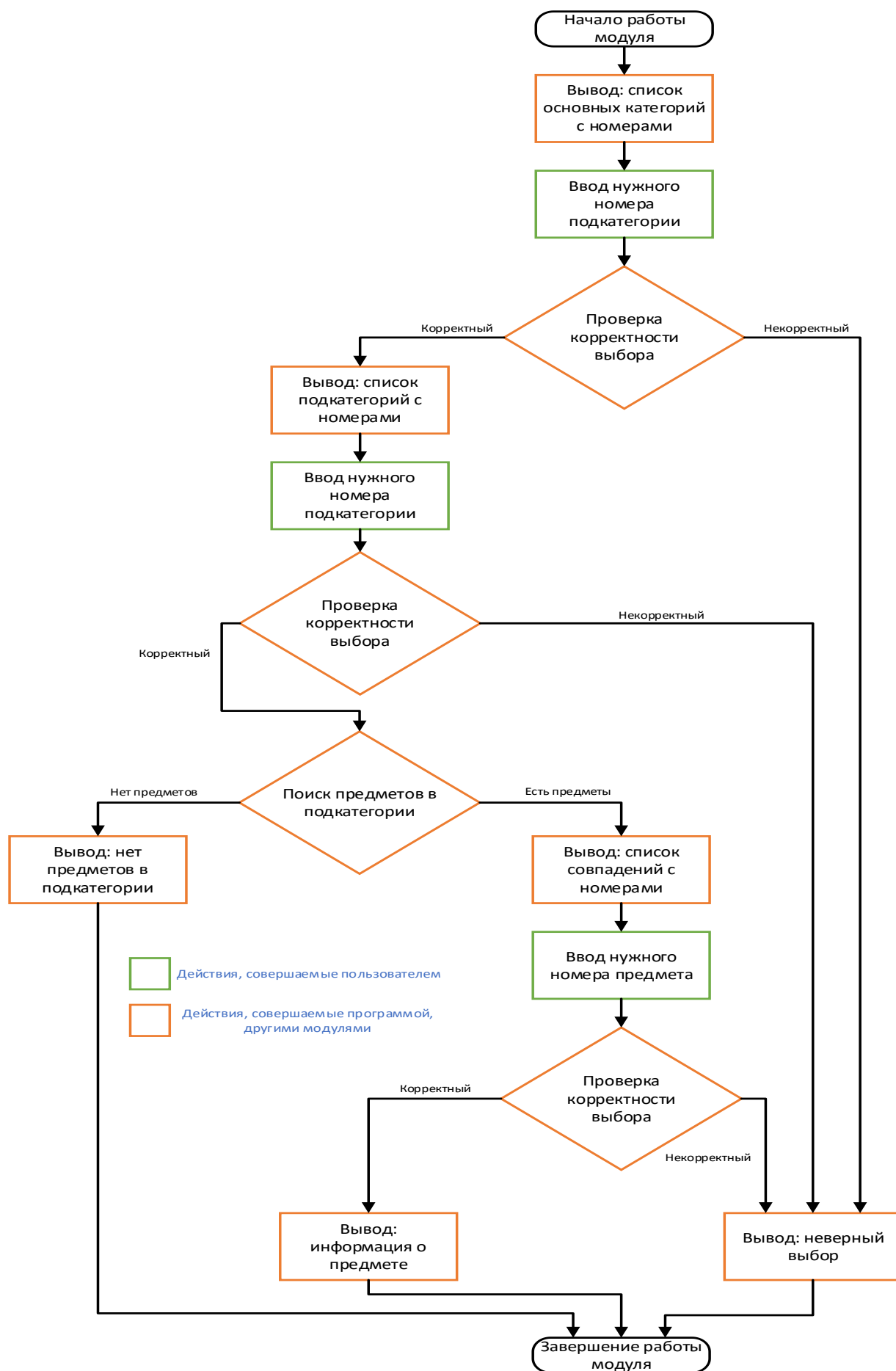
### 1.2.1. Алгоритм нечеткого поиска

Поиск по названию реализован через механизм частичного совпадения строк (substring matching). Пользовательский ввод нормализуется путем приведения к нижнему регистру и удаления лишних пробелов, после чего происходит поиск всех предметов, содержащих введенную подстроку.

Такой подход позволяет находить предметы даже при наличии опечаток или неполном знании точного названия. (Изобр. 3).



*Изобр. 3. Пример поиска предмета по названию*



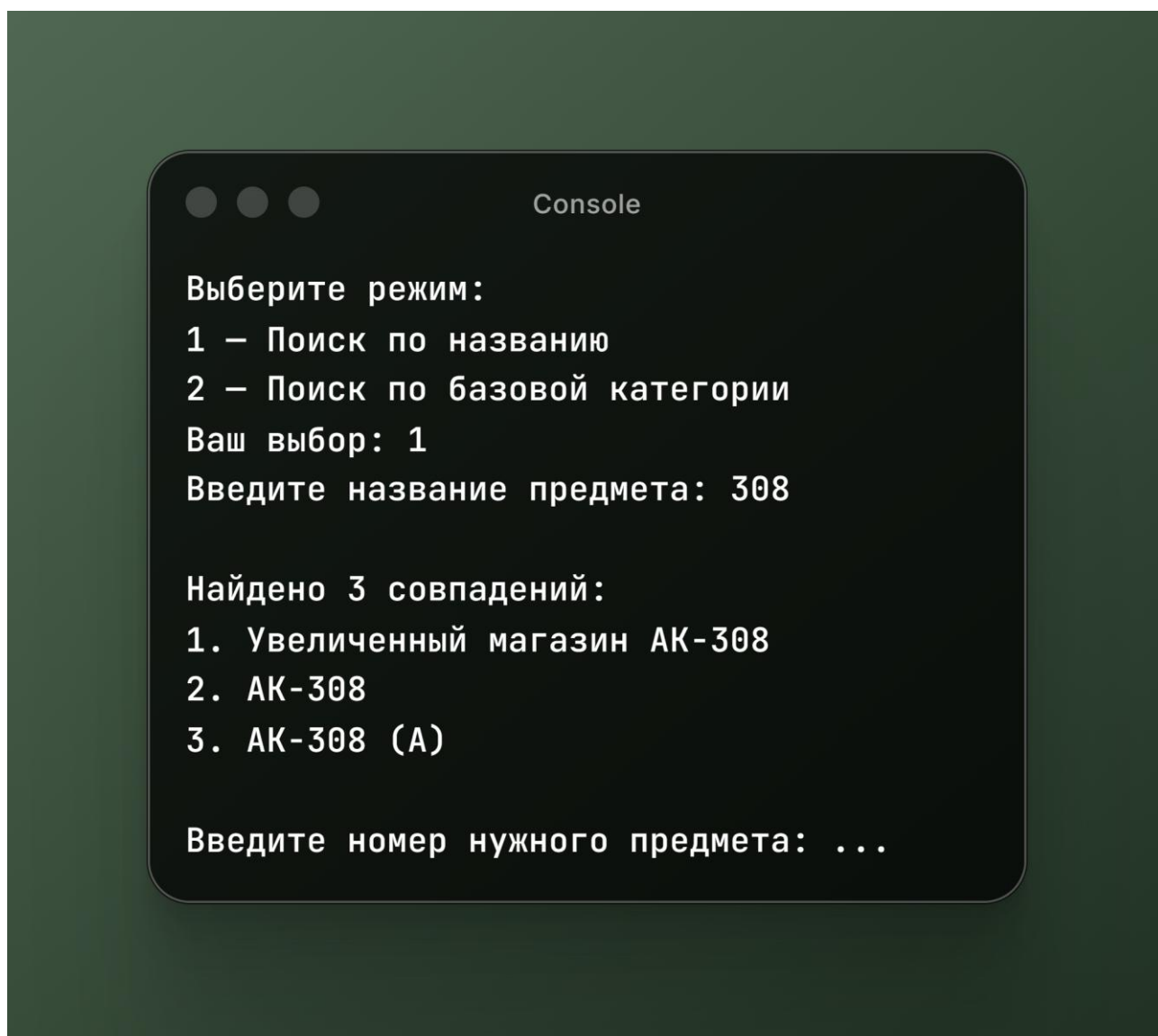
Блок-схема 2. Общая схема работы поиска предмета по названию



### 1.2.1. Алгоритм нечеткого поиска

Поиск по названию реализован через механизм частичного совпадения строк (substring matching). Пользовательский ввод нормализуется путем приведения к нижнему регистру и удаления лишних пробелов, после чего происходит поиск всех предметов, содержащих введенную подстроку.

Такой подход позволяет находить предметы даже при наличии опечаток или неполном знании точного названия. (Изобр. 3).



*Изобр. 3. Пример поиска предмета по названию*

### 1.2.2. Многоуровневая система обработки результатов

В зависимости от количества найденных совпадений система применяет различные сценарии взаимодействия:

- Нулевые результаты - пользователю выводится сообщение "Ничего не найдено" с рекомендацией уточнить критерии поиска.
- Единственное совпадение - система автоматически переходит к отображению детальной информации о предмете.
- Множественные совпадения - формируется нумерованный список, из которого пользователь выбирает нужный вариант.

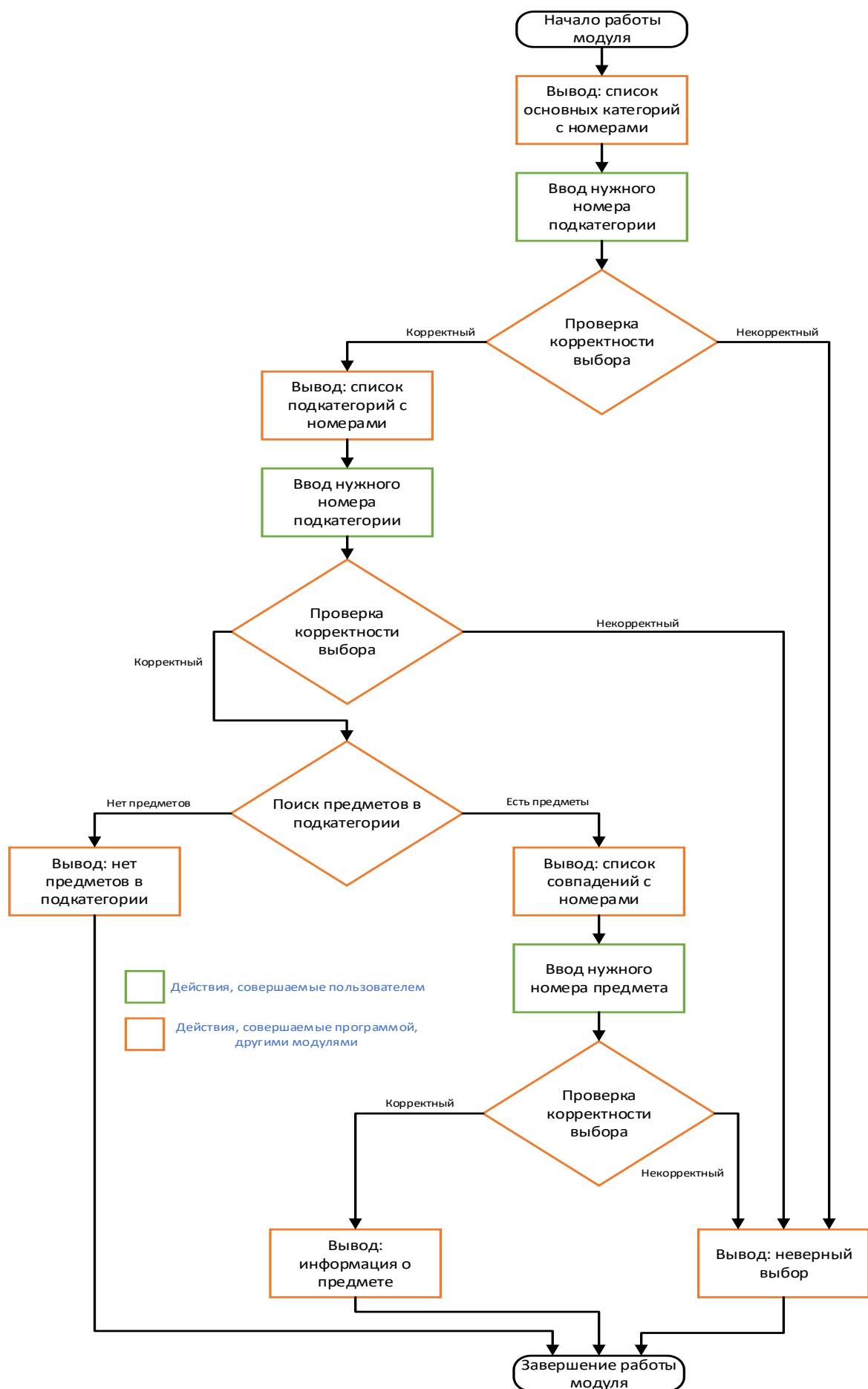
Для обработки пользовательского выбора реализована система проверки корректности ввода, включающая защиту от выхода за границы списка и обработку нечисловых значений.

### **1.3. Иерархическая система поиска по категориям**

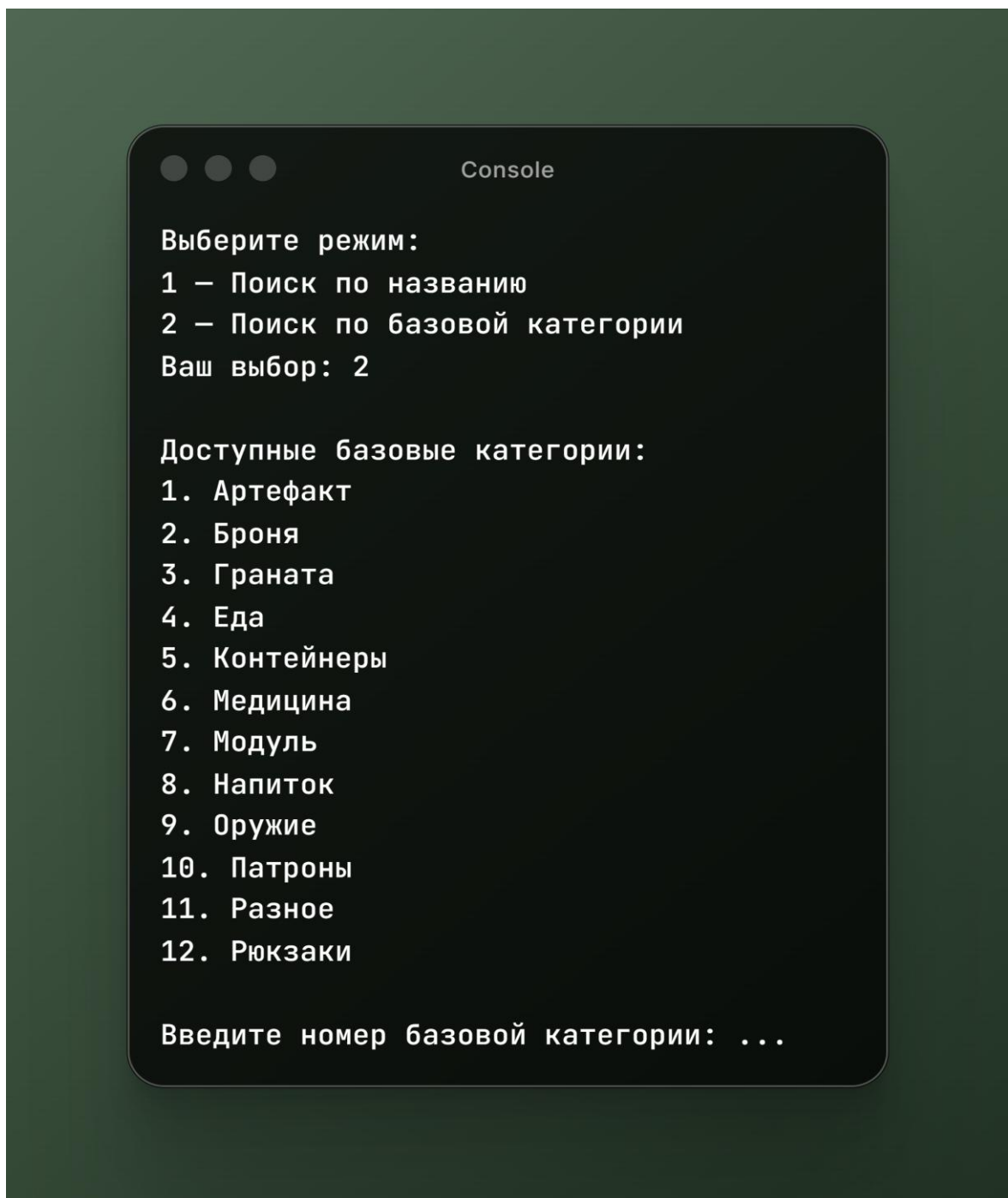
Иерархическая система поиска по категориям представляет собой многоуровневую структуру, обеспечивающую навигацию от базовых категорий к подкатегориям и предметам. Архитектура реализована через последовательность взаимосвязанных функций, каждая из которых отвечает за извлечение данных из CSV-файла и их фильтрацию. Графическое изображение данной системы представлено на (блок-схема 3).

#### **1.3.1. Построение древовидной структуры категорий**

Система категорий организована по иерархическому принципу с поддержкой многоуровневой вложенности. На верхнем уровне находятся базовые категории, которые извлекаются через анализ первого уровня вложенности (Изобр.4). Особенностью реализации является фильтрация категории "другое", что позволяет сохранить чистоту и релевантность основного каталога.



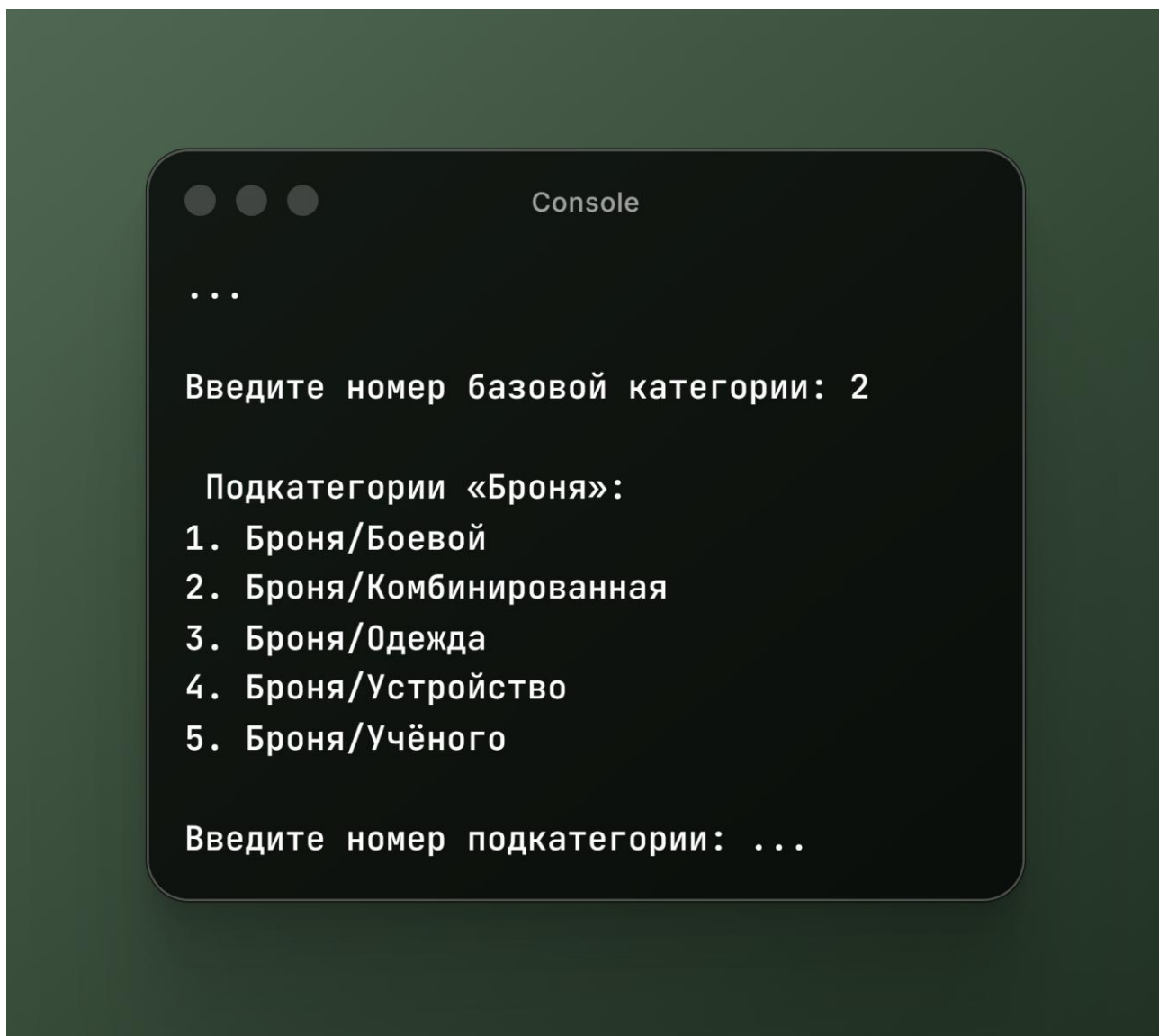
Блок-схема 3. Общая схема работы поиска предмета по категории



*Изобр. 4. Выбор категории для поиска предмета*

### **1.3.2. Навигация по подкатегориям**

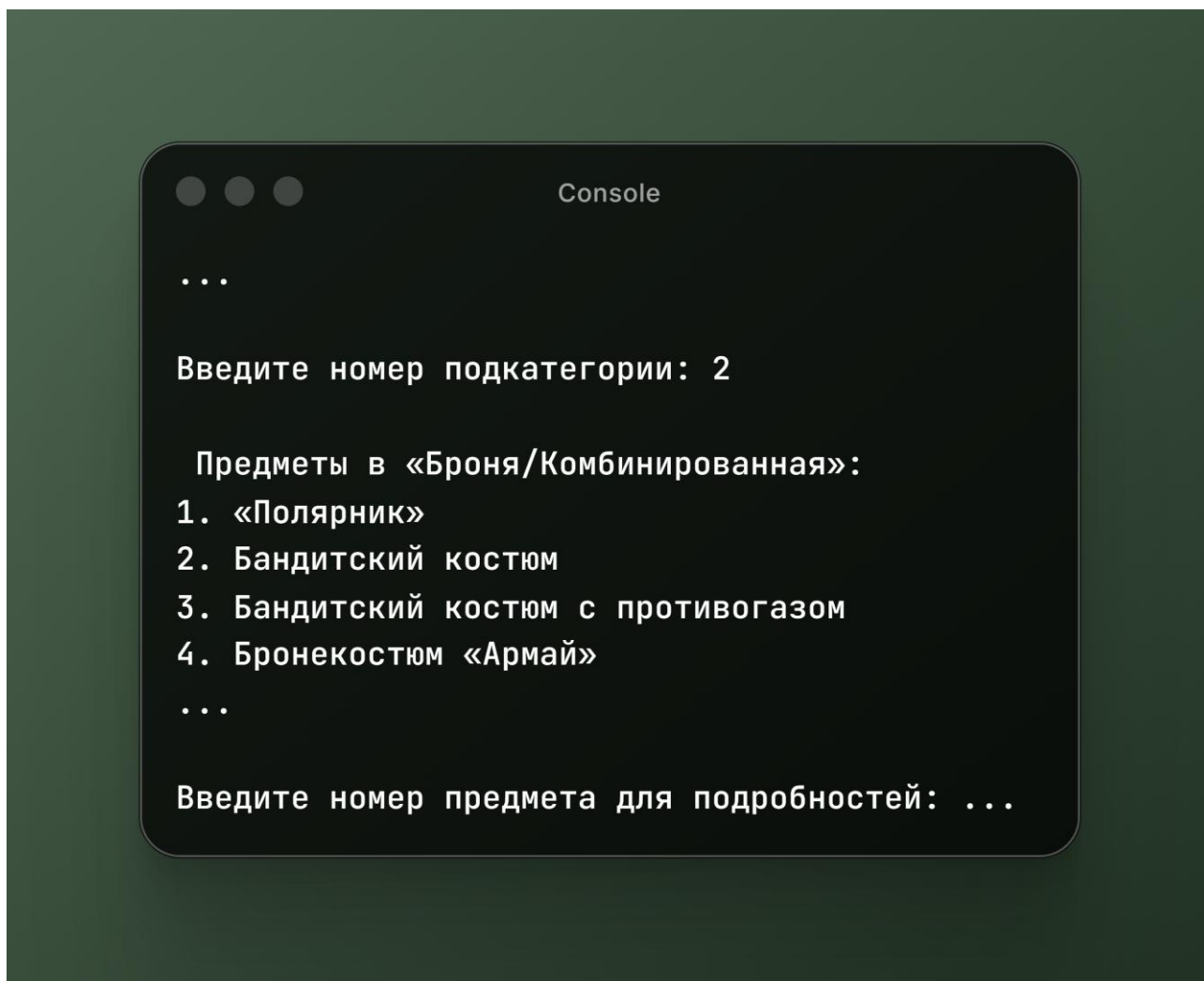
После выбора базовой категории система извлекает все связанные подкатегории через поиск по префиксу. Этот подход обеспечивает динамическое построение меню навигации без жесткой привязки к фиксированной структуре категорий (Изобр. 5).



*Изобр. 5. Выбор подкатегории для поиска*

### **1.3.3. Поиск предметов в категориях**

Для каждой подкатегории система выполняет поиск всех относящихся к ней предметов, исключая элементы из категории "другое" (Изобр. 6).



*Изобр. 6. Выбор предмета из списка*

## **1.4. Детальный парсинг информации о предметах**

### **1.4.1. Извлечение базовых характеристик**

Для получения детальной информации о конкретном предмете система загружает его персональную страницу по URL вида <https://stcx.ru/item/{ID}>. Использование BeautifulSoup позволяет эффективно извлекать структурированные данные из HTML. Извлекаются следующие ключевые элементы:

- Название - из заголовка h2 с классом item-title
- Описание - из параграфа с классом item-description
- Ранг - путем поиска в блоках свойств с классом property-item
- Изображение - из тега img с классом page-item-image

### **1.4.2. Парсинг аукционных лотов с использованием Selenium**

Для получения информации об активных лотах требуется более сложный подход, так как эти данные часто загружаются динамически через JavaScript. Решение использует headless-браузер Chrome через Selenium WebDriver. Критически важные аспекты реализации:

- Headless-режим - позволяет запускать браузер без графического интерфейса
- Отключение GPU - повышает стабильность в серверных окружениях
- Таймаут загрузки - обеспечивает полную загрузку динамического контента
- Целевое извлечение - получение только нужного блока через JavaScript

### **1.4.3. Анализ и ранжирование лотов**

Извлеченные данные о лотах проходят многоэтапную обработку:

- Парсинг структуры - извлечение отдельных лотов по классу
- Извлечение параметров - поиск цен ставки, выкупа и времени для каждого лота
- Сортировка по цене - автоматическое упорядочивание по возрастанию цены выкупа
- Ограничение вывода - возврат только топ-5 самых дешевых предложений

## **1.5. Обработка ошибок и обеспечение надежности**

### **1.5.1. Многоуровневая система обработки исключений**

Система предусматривает обработку различных типов ошибок:

- Сетевые проблемы - таймауты и ошибки соединения при обращении к API
- Парсинг ошибок - изменения в структуре HTML страниц
- Пользовательский ввод - некорректные номера и несуществующие выборы
- Файловые операции - проблемы с доступом к CSV-файлу

## **1.6. Производительность и оптимизация**

### **1.6.1. Методы снижения нагрузки**

Система использует несколько подходов для минимизации нагрузки на сервер, на котором развернута программа:

- Локальное кэширование - хранение каталога предметов в CSV

- Пакетная обработка - массовое извлечение данных при инициализации
- Эффективные запросы - целевое получение только необходимой информации
- Оптимальные таймауты - баланс между полнотой данных и скоростью работы

### **1.6.2. Оптимизация алгоритмов поиска**

Использование сортировки данных при сохранении позволяет применять эффективные алгоритмы поиска с минимальными временными затратами. Регистронезависимый поиск и нормализация строк обеспечивают высокую релевантность результатов при сохранении производительности.

Разработанная система парсинга демонстрирует высокую эффективность в условиях динамически изменяющихся данных аукциона, обеспечивая пользователей актуальной и структурированной информацией для принятия взвешенных решений при участии в торгах.



## ГЛАВА 2. ИНТЕГРАЦИЯ С ОФИЦИАЛЬНЫМ API STALCRAFT

### 2.1. Архитектура и общая схема работы API клиента

Модуль клиента API представляет собой специализированный программный компонент, реализованный в файле `bot_api_client.py` и обеспечивающий взаимодействие с официальным программным интерфейсом игры Stalcraft для получения актуальных данных об аукционных лотах в режиме реального времени. Архитектура модуля построена по принципам инкапсуляции и модульности, что позволяет скрыть детали сетевого взаимодействия от остальной системы и предоставить простой высокоуровневый интерфейс для работы с данными аукциона.

Основной процесс работы модуля начинается с инициализации клиента с указанием учетных данных приложения (идентификатора клиента и секретного ключа), после чего следует процесс аутентификации, формирование HTTP-запросов к конечным точкам API, обработка полученных ответов и преобразование данных в удобный для дальнейшей обработки формат.

#### 2.1.1. Инициализация клиента

При создании экземпляра класса `StalcraftAPI` происходит инициализация ключевых параметров соединения с API.

В ходе инициализации устанавливаются следующие конфигурационные параметры:

| Параметр                   | Значение                                  | Назначение                               |
|----------------------------|-------------------------------------------|------------------------------------------|
| <code>client_id</code>     | Идентификатор приложения                  | Аутентификация клиента в OAuth 2.0       |
| <code>client_token</code>  | Секретный ключ                            | Аутентификация клиента в OAuth 2.0       |
| <code>region</code>        | Регион сервера (по умолчанию "eu")        | Выбор региона для запросов к API         |
| <code>base_url</code>      | <code>https://eapi.stalcraft.net</code>   | Базовый URL API                          |
| <code>auth_url</code>      | <code>https://exbo.net/oauth/token</code> | URL сервера авторизации                  |
| <code>timeout</code>       | 10 секунд                                 | Таймаут для HTTP-запросов                |
| <code>_token_buffer</code> | 60 секунд                                 | Буфер безопасности для обновления токена |

Таблица 1. Конфигурационные параметры API клиента

Важной особенностью реализации является наличие буфера безопасности (`_token_buffer`), который составляет 60 секунд. Этот буфер используется для того, чтобы запросить новый токен доступа до истечения срока действия текущего, что предотвращает ситуации, когда запрос к API может быть отклонен из-за использования просроченного токена.

### **2.1.2. Жизненный цикл взаимодействия с API**

Взаимодействие с API представляет собой последовательность этапов, каждый из которых обеспечивает надежность и корректность обмена данными.

Процесс включает следующие этапы:

1. **Проверка актуальности токена** - система определяет, необходим ли запрос нового токена доступа или можно использовать существующий.
2. **Аутентификация** - при необходимости выполняется запрос нового токена через OAuth 2.0.
3. **Формирование запроса** - подготовка HTTP-запроса с необходимыми заголовками и параметрами.
4. **Отправка запроса** - выполнение HTTP-запроса к конечной точке API.
5. **Обработка ответа** - анализ полученного ответа и обработка возможных ошибок.
6. **Преобразование данных** - конвертация JSON-ответа во внутреннюю структуру данных.
7. **Возврат результата** - передача обработанных данных вызывающему коду.

## **2.2. Механизм аутентификации OAuth 2.0**

Доступ к защищенным методам официального API осуществляется с использованием стандарта OAuth 2.0, который обеспечивает безопасную авторизацию клиентских приложений. Этот стандарт является промышленным решением для делегирования доступа и широко используется в современных веб-сервисах.

### **2.2.1. Процесс получения токена доступа**

Процесс аутентификации представляет собой многоэтапную процедуру, которая включает следующие этапы:

## Этап 1: Проверка кэшированного токена

Перед выполнением любого запроса к API система проверяет наличие актуального токена доступа в оперативной памяти. Проверка осуществляется путем сравнения текущего времени с временем истечения токена с учетом буфера безопасности. Такой подход позволяет минимизировать количество запросов к серверу авторизации и значительно сократить время отклика системы.

## Этап 2: Формирование запроса авторизации

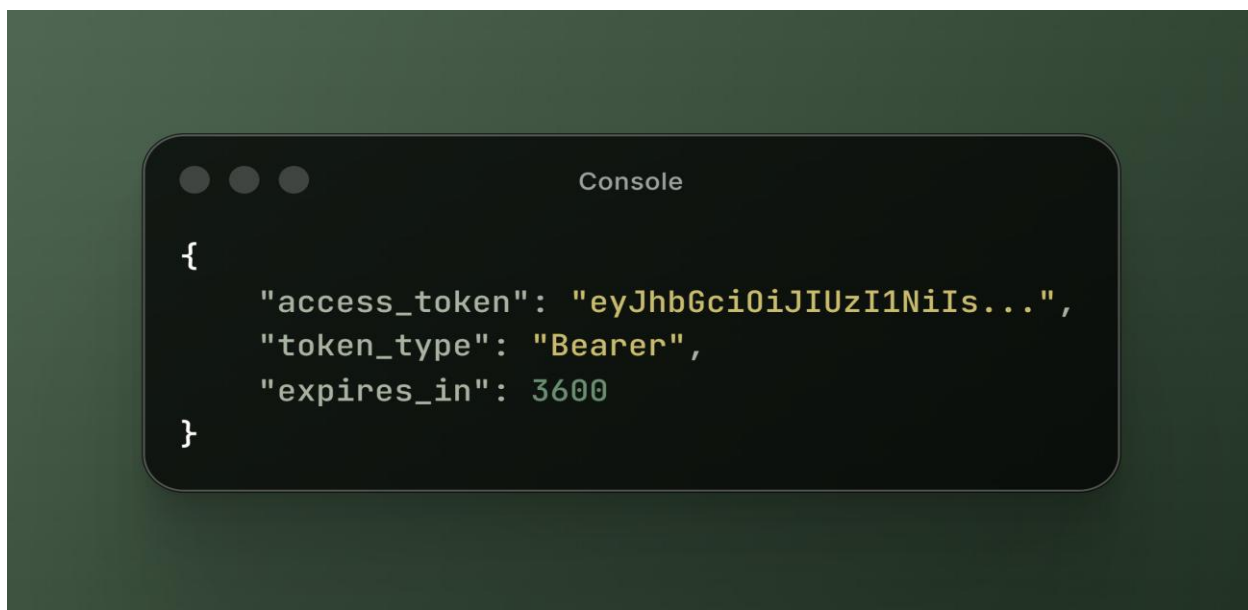
При отсутствии актуального токена система формирует POST-запрос к эндпоинту `https://exbo.net/oauth/token` с параметрами в формате `application/x-www-form-urlencoded`:

| Параметр                   | Значение                        | Назначение                     |
|----------------------------|---------------------------------|--------------------------------|
| <code>grant_type</code>    | <code>client_credentials</code> | Тип предоставляемых полномочий |
| <code>client_id</code>     | Идентификатор приложения        | Идентификация клиента          |
| <code>client_secret</code> | Секретный ключ                  | Проверка подлинности клиента   |

Таблица 2. Параметры запроса авторизации

## Этап 3: Обработка ответа сервера авторизации

При успешной аутентификации сервер возвращает JSON-объект со следующей структурой, представленной ниже: (Изобр. 7)



Изображение 7. Ответ сервера авторизации

Система извлекает токен доступа и время его жизни, после чего сохраняет эти значения в атрибутах объекта клиента для последующего использования.

### 2.2.2. Управление временем жизни токена

Критически важным аспектом реализации является корректное управление временем жизни токена доступа. Система реализует следующий алгоритм:

1. **Сохранение времени истечения** - при получении токена вычисляется абсолютное время истечения путем добавления значения `expires_in` к текущему времени.
2. **Буфер безопасности** - токен считается устаревшим за 60 секунд до фактического истечения, что предотвращает использование просроченного токена.
3. **Автоматическое обновление** - при обнаружении устаревшего токена система автоматически запрашивает новый без вмешательства пользователя.

Данный подход обеспечивает непрерывность работы системы даже при длительных сессиях взаимодействия с API.

### 2.2.3. Обработка ошибок авторизации

Система предусматривает обработку различных типов ошибок, возникающих в процессе аутентификации (Таблица 3).

| Код состояния | Описание                | Действие системы                                     |
|---------------|-------------------------|------------------------------------------------------|
| 200           | Успешная аутентификация | Сохранение токена и продолжение работы               |
| 401           | Неверные учетные данные | Генерация исключения <code>StalcraftAPIError</code>  |
| 403           | Доступ запрещен         | Генерация исключения с рекомендацией проверить права |
| 429           | Превышен лимит запросов | Информирование о необходимости ожидания              |
| 5xx           | Ошибка сервера          | Логирование и повторная попытка                      |

Таблица 3. Обработка ошибок авторизации

При возникновении сетевых ошибок (таймауты, ошибки соединения) система генерирует специализированное исключение `StalcraftAPIError` с

описанием проблемы, что позволяет вышестоящему коду корректно обработать ситуацию и информировать пользователя.

## 2.3. Формирование HTTP-запросов и обработка ответов

### 2.3.1. Структура HTTP-запроса

Каждый запрос к API формируется с использованием стандартных HTTP-методов и включает необходимые заголовки для аутентификации и указания формата данных.

Обязательные заголовки запроса:

| Заголовок     | Значение              | Назначение              |
|---------------|-----------------------|-------------------------|
| Authorization | Bearer {access_token} | Аутентификация запроса  |
| Content-Type  | application/json      | Формат тела запроса     |
| Accept        | application/json      | Ожидаемый формат ответа |

Таблица 4. Обязательные заголовки HTTP-запроса

### 2.3.2. Универсальный метод выполнения запросов

Для обеспечения единообразия обработки всех запросов к API реализован универсальный метод `_request`, который инкапсулирует всю логику взаимодействия с сервером.

Метод выполняет следующие функции:

1. **Формирование полного URL** - объединение базового URL с указанным эндпоинтом.
2. **Добавление заголовков** - включение заголовков аутентификации и дополнительных заголовков, переданных вызывающим кодом.
3. **Установка таймаута** - применение значения таймаута по умолчанию, если оно не было явно указано.
4. **Выполнение запроса** - отправка HTTP-запроса с использованием указанного метода (GET, POST и т.д.).
5. **Обработка ответа** - анализ кода состояния HTTP и соответствующая реакция.
6. **Повторная попытка при ошибке 401** - при получении ошибки авторизации система сбрасывает кэшированный токен, запрашивает новый и повторяет запрос.

7. **Возврат результата** - преобразование JSON-ответа в словарь Python и возврат вызывающему коду.

### 2.3.3. Обработка кодов состояния HTTP

Система реализует многоуровневую обработку различных кодов состояния HTTP, что обеспечивает надежность работы и информативность сообщений об ошибках (Таблица 5).

| Код состояния | Тип ошибки         | Обработка                                     |
|---------------|--------------------|-----------------------------------------------|
| 200           | Успех              | Возврат JSON-ответа                           |
| 400           | Ошибка клиента     | Генерация исключения с описанием ошибки       |
| 401           | Ошибка авторизации | Сброс токена и повторная попытка              |
| 404           | Ресурс не найден   | Генерация исключения с указанием эндпоинта    |
| 429           | Превышение лимита  | Генерация исключения о rate limit             |
| 5xx           | Ошибка сервера     | Генерация исключения с кодом и текстом ошибки |

Таблица 5. Обработка кодов состояния HTTP

Особое внимание уделено обработке ошибки 401 (Unauthorized). При получении этого кода система выполняет следующие действия:

1. Сбрасывает кэшированный токен доступа
2. Запрашивает новый токен через метод
3. Повторяет исходный запрос с новым токеном.

Такой механизм обеспечивает автоматическое восстановление работоспособности при отзыве токена на стороне сервера.

### 2.3.4. Обработка сетевых ошибок

Помимо обработки кодов состояния HTTP, система предусматривает обработку сетевых ошибок, которые могут возникать при проблемах с соединением (Таблица 6):

| Тип ошибки | Описание                    | Действие системы                       |
|------------|-----------------------------|----------------------------------------|
| Timeout    | Превышение времени ожидания | Генерация исключения "Таймаут запроса" |

|                   |                       |                                                    |
|-------------------|-----------------------|----------------------------------------------------|
| ConnectionError   | Ошибка подключения    | Генерация исключения<br>"Ошибка подключения к API" |
| Другие исключения | Непредвиденные ошибки | Генерация исключения с описанием                   |

Таблица 6. Обработка сетевых ошибок

Все исключения наследуются от базового класса StalcraftAPIError, что позволяет вышестоящему коду единообразно обрабатывать все ошибки, связанные с взаимодействием с API.

## 2.4. Получение данных об аукционных лотах

### 2.4.1. Метод получения списка лотов

Основным методом работы с аукционом является метод, который позволяет получить список активных лотов для заданного предмета.

Метод принимает следующие параметры:

| Параметр   | Тип     | Обязательный | Описание               | Допустимые значения                                  |
|------------|---------|--------------|------------------------|------------------------------------------------------|
| item_id    | String  | Да           | Идентификатор предмета | ID из базы данных предметов                          |
| limit      | Integer | Нет          | Количество лотов       | 1-200 (по умолчанию 5)                               |
| offset     | Integer | Нет          | Смещение для пагинации | 0 и более                                            |
| sort       | String  | Нет          | Поле сортировки        | time_created, time_left, current_price, buyout_price |
| order      | String  | Нет          | Порядок сортировки     | asc, desc                                            |
| additional | Boolean | Нет          | Расширенная информация | true, false                                          |

Таблица 7. Параметры метода получения лотов

### 2.4.2. Валидация параметров

Перед выполнением запроса система выполняет валидацию входных параметров, что предотвращает некорректные запросы к API и обеспечивает предсказуемость поведения системы:

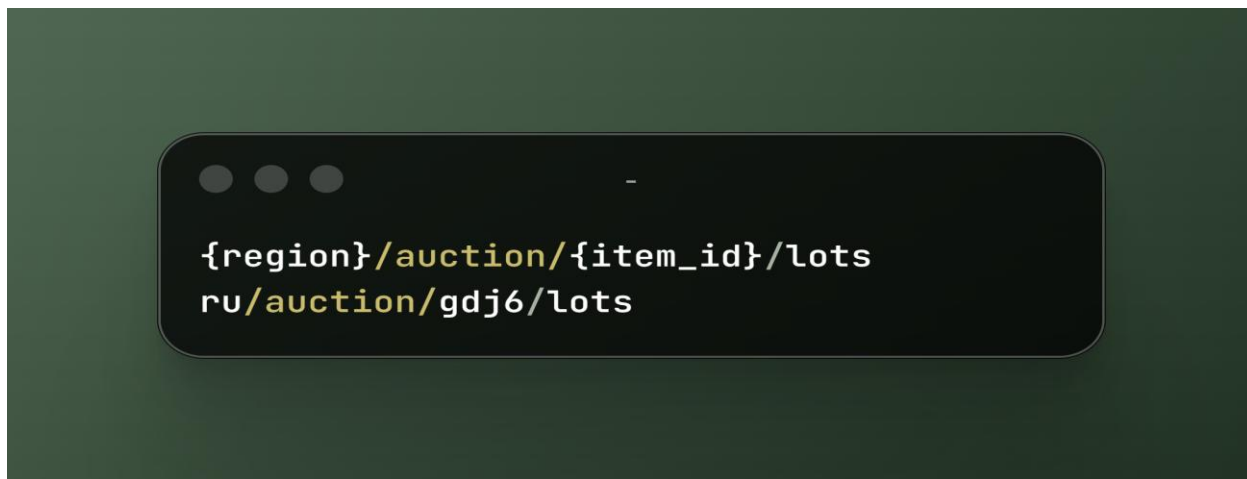
1. **Ограничение значения limit** - значение приводится к диапазону от 1 до 200 с использованием функций `max()` и `min()`.
2. **Проверка неотрицательности offset** - значение не может быть меньше нуля.
3. **Проверка допустимых значений sort** - поле сортировки должно быть одним из четырех разрешенных значений.
4. **Проверка значения order** - порядок сортировки должен быть либо `asc`, либо `desc`.

При нарушении любого из этих условий система генерирует исключение `ValueError` с описанием допустимых значений, что позволяет разработчику быстро выявить и исправить ошибку.

#### 2.4.3. Формирование эндпоинта

Эндпоинт для запроса лотов формируется динамически на основе идентификатора предмета и региона (Изобр. )

Например, для предмета с ID `gdj6` в регионе `ru` эндпоинт будет иметь вид (Изобр. 8):



Изображение 8. Формирование эндпоинта

Такой подход обеспечивает гибкость системы и возможность работы с различными регионами серверов игры.

#### 2.4.4. Структура ответа API

Ответ API содержит информацию о лотах в формате JSON со следующей структурой (Изобр. 9):



```
{
  "total": 42,
  "lots": [
    {
      "itemId": "lot_123",
      "amount": 5,
      "startPrice": 10000,
      "currentPrice": 15000,
      "buyoutPrice": 20000,
      "startTime": "2025-01-15T10:00:00Z",
      "endTime": "2025-01-15T18:00:00Z"
    }
  ]
}
```

*Изображение 9. ответ API*

Система возвращает этот ответ вызывающему коду без дополнительной обработки, что обеспечивает максимальную гибкость при работе с данными.

## **2.5. Обработка пагинации**

### **2.5.1. Проблема пагинации в API**

Официальный API Stalcraft возвращает данные об аукционных лотах ограниченными порциями (страницами). Максимальное количество лотов, возвращаемых в одном запросе, составляет 200 записей. Для предметов с большим количеством активных лотов это означает необходимость выполнения нескольких последовательных запросов для получения полного списка.

### **2.5.2. Алгоритм автоматической пагинации**

Для решения проблемы пагинации реализован метод `get_all_lots`, который автоматически выполняет все необходимые запросы и объединяет результаты в единый список.

Алгоритм работает следующим образом:

1. **Инициализация** - создается пустой список для хранения всех лотов, устанавливается начальное смещение `offset = 0` и максимальный размер страницы `limit = 200`.
2. **Цикл запросов** - система выполняет запросы к API в цикле, каждый раз увеличивая значение `offset` на количество полученных лотов.
3. **Проверка завершения** - цикл продолжается до тех пор, пока количество полученных лотов меньше значения `limit`, что свидетельствует о достижении последней страницы.
4. **Объединение результатов** - все полученные лоты добавляются в общий список с использованием метода `extend()`.
5. **Возврат результата** - после получения всех лотов метод возвращает полный список вызывающему коду.

### 2.5.3. Оптимизация запросов

Для минимизации нагрузки на сервер и ускорения получения данных применяются следующие оптимизации:

1. **Максимальный размер страницы** - использование значения `limit = 200` позволяет получить максимальное количество лотов за один запрос, сокращая общее количество обращений к API.
2. **Сортировка на стороне сервера** - параметр `sort` позволяет получить данные уже отсортированными, что исключает необходимость сортировки на стороне клиента.
3. **Отключение дополнительной информации** - параметр `additional = False` отключает загрузку расширенной информации о лотах, когда она не требуется, что уменьшает размер ответа и ускоряет передачу данных.

## 2.6. Дополнительные методы работы с данными

### 2.6.1. Поиск самого дешевого лота

Для упрощения работы с данными реализован метод `find_cheapest_lot`, который находит самый дешевый лот для заданного предмета.

Метод принимает два параметра:

- `item_id` - идентификатор предмета
- `use_buyout` - флаг, определяющий, использовать ли цену выкупа (`buyoutPrice`) или текущую цену (`currentPrice`)

Алгоритм работает следующим образом:

1. Получение всех лотов для предмета с сортировкой по текущей цене.
2. Проверка наличия лотов - если список пуст, возвращается None.
3. Поиск минимального значения - использование функции `min()` с ключом, определяющим поле для сравнения.
4. Обработка отсутствующих значений - если цена выкупа отсутствует, используется текущая цена, и наоборот.

### 2.6.2. Получение информации о предмете

Метод `get_item_info` позволяет получить детальную информацию о конкретном предмете по его идентификатору. Этот метод формирует запрос к эндпоинту `{region}/items/{item_id}` и возвращает JSON-ответ с данными о предмете.

### 2.6.3. Форматирование временных меток

Для удобства отображения информации о лотах реализован метод `format_time`, который преобразует временные метки из формата ISO 8601 в человекочитаемый вид.

Метод выполняет следующие действия:

1. **Замена суффикса** - символ `Z` (обозначающий часовой пояс UTC) заменяется на `+00:00` для совместимости с функцией `fromisoformat()`.
2. **Парсинг даты** - преобразование строки в объект `datetime`.
3. **Форматирование** - вывод даты в формате ДД.ММ ЧЧ:ММ (например, 15.01 14:30).
4. **Обработка ошибок** - при возникновении исключения возвращается исходная строка без преобразования.

Такой подход обеспечивает надежность работы метода даже при получении некорректных данных от API.

## 2.7. Обработка ошибок и обеспечение надежности

### 2.7.1. Иерархия исключений

Система использует специализированный класс исключений `StalcraftAPIError`, который наследуется от базового класса `Exception`. Это позволяет вышестоящему коду единообразно обрабатывать все ошибки, связанные с взаимодействием с API.

### 2.7.2. Типы ошибок и стратегии обработки

Система предусматривает обработку различных типов ошибок, каждая из которых требует своей стратегии обработки (Таблица 8).

| Тип ошибки         | Причина возникновения                         | Стратегия обработки                                  |
|--------------------|-----------------------------------------------|------------------------------------------------------|
| Ошибки авторизации | Неверные учетные данные, истечение токена     | Автоматическое обновление токена, повторная попытка  |
| Сетевые ошибки     | Таймауты, проблемы с соединением              | Логирование, информирование пользователя             |
| Ошибки валидации   | Некорректные параметры запроса                | Генерация исключения с описанием допустимых значений |
| Ошибки API         | Ошибки на стороне сервера, превышение лимитов | Логирование, повторная попытка или информирование    |
| Ошибки парсинга    | Некорректный формат ответа                    | Логирование, возврат исходных данных                 |

Таблица 8. Типы ошибок и стратегии обработки

### 2.7.3. Механизм повторных попыток

Для повышения надежности работы системы реализован механизм повторных попыток при возникновении временных ошибок. Этот механизм особенно важен для обработки ошибок авторизации (код 401), когда система автоматически запрашивает новый токен и повторяет запрос.

### 2.7.4. Логирование ошибок

Логирование всех действий в системе реализовано через стандартный модуль logging языка Python. Для обеспечения надежности, оперативного мониторинга и удобства последующего анализа используется три типа обработчиков событий: файловый (FileHandler), консольный (StreamHandler) и кастомный (DBLogHandler). Кастомный обработчик отвечает за структурированную запись событий в отдельную базу данных logs.db с помощью SQLite для долгосрочного хранения и аудита (Изобр. 24).

Такой многоуровневый подход позволяет одновременно выводить критические ошибки в консоль для быстрой диагностики, сохранять полную историю в текстовый файл и формировать структурированный журнал в базе данных для программного анализа и построения отчетов.

## 2.8. Производительность и оптимизация

### 2.8.1. Методы оптимизации производительности

Система использует несколько подходов для обеспечения высокой производительности при работе с API (Таблица 9).

| Метод оптимизации                    | Реализация                                  | Эффект                                               |
|--------------------------------------|---------------------------------------------|------------------------------------------------------|
| Кэширование токена                   | Хранение токена в оперативной памяти        | Сокращение количества запросов к серверу авторизации |
| Буфер безопасности                   | Обновление токена за 60 секунд до истечения | Предотвращение использования просроченного токена    |
| Максимальный размер страницы         | Использование limit = 200                   | Сокращение количества запросов при пагинации         |
| Отключение дополнительной информации | Параметр additional = False                 | Уменьшение размера ответа                            |
| Сортировка на стороне сервера        | Параметр sort                               | Исключение необходимости сортировки на клиенте       |
| Таймауты запросов                    | Значение timeout = 10 секунд                | Предотвращение зависания при проблемах с сетью       |

Таблица 9. Методы оптимизации производительности

### 2.8.2. Баланс между полнотой данных и скоростью работы

Критически важным аспектом проектирования API клиента является нахождение баланса между полнотой получаемых данных и скоростью работы системы. Этот баланс достигается за счет:

1. **Выборочной загрузки данных** - получение только необходимой информации через параметры запроса.
2. **Ленивой загрузки** - загрузка детальной информации о предметах только по запросу пользователя.
3. **Кэширования на уровне приложения** - использование декоратора @lru\_cache для кэширования результатов частых запросов (описано в Главе 3).

### 2.8.3. Масштабируемость

Архитектура API клиента позволяет легко масштабировать функциональность:

1. **Добавление новых методов** - реализация новых методов для работы с другими эндпоинтами API не требует изменения существующего кода.
2. **Поддержка новых регионов** - параметр region позволяет работать с различными регионами серверов игры.
3. **Интеграция с другими системами** - модульная архитектура позволяет использовать API клиент в других проектах без изменений.

## 2.9. Заключение по главе

Разработанный модуль API клиента представляет собой законченное решение для взаимодействия с официальным программным интерфейсом игры Stalcraft. Система успешно сочетает в себе:

1. **Надежность** - многоуровневая обработка ошибок, автоматическое обновление токена, механизмы повторных попыток.
2. **Производительность** - кэширование токена, оптимизация запросов, эффективная обработка пагинации.
3. **Безопасность** - использование стандарта OAuth 2.0, защита учетных данных, буфер безопасности.
4. **Гибкость** - модульная архитектура, поддержка различных регионов, расширяемость.
5. **Удобство использования** - простой высокоуровневый интерфейс, валидация параметров, информативные сообщения об ошибках.

Интеграция с основным модулем бота (описанным в Главе 3) обеспечивает получение актуальных данных об аукционных лотах в реальном времени, что является критически важным для функционирования всей системы в целом. Модуль готов к промышленному использованию и может быть легко расширен дополнительным функционалом по мере необходимости. Взаимодействие с подсистемой хранения данных (описанной в Главе 4) обеспечивает сохранение и обработку полученной информации, что завершает полный цикл работы системы от получения данных до их представления пользователю.

## ГЛАВА 3. ТЕЛЕГРАММ-БОТ ДЛЯ ПОИСКА ПРЕДМЕТОВ НА АУКЦИОНЕ STALCRAFT

### 3.1. Архитектура и общая схема работы бота

Система Telegram-бота представляет собой многоуровневую архитектуру, построенную на асинхронной модели работы с использованием библиотеки `aiogram 3.28.2`. Бот функционирует как посредник между пользователем и системой парсинга данных, предоставляя удобный интерфейс для доступа к информации об аукционных предметах. Разработанная система представляет собой многокомпонентное приложение, состоящее из нескольких взаимосвязанных модулей, каждый из которых отвечает за определенный аспект функционирования.

Система включает три основных программных модуля:

1. **Модуль импорта данных (`db_importer.py`)** - самостоятельная программа, отвечающая за получение, обработку и сохранение справочной информации о предметах игры. Этот модуль работает независимо от основного бота и запускается по мере необходимости для обновления базы данных предметов.
2. **Модуль клиента API (`bot_api_client.py`)** - библиотека, обеспечивающая взаимодействие с официальным программным интерфейсом игры Stalcraft для получения актуальных данных об аукционных лотах в реальном времени.
3. **Основной модуль бота (`bot.py`)** - ядро системы, реализующее пользовательский интерфейс, обработку команд, управление диалогами и координацию работы всех компонентов.

#### 3.1.1. Принцип работы основного модуля

При запуске основного модуля происходит следующая последовательность действий:

1. **Инициализация подсистемы хранения данных** - создание соединений с базами данных пользователей и журнала событий, проверка целостности структуры таблиц.
2. **Загрузка справочных данных** - чтение информации о предметах из предварительно созданной базы данных (`items.db`), которая была сформирована модулем импорта.
3. **Загрузка пользовательских данных** - получение списка авторизованных пользователей в оперативную память для ускорения проверки прав доступа.

4. **Настройка подсистемы регистрации событий** - инициализация механизмов записи логов в файл и базу данных для последующего анализа и диагностики.
5. **Запуск механизма обработки событий** - переход в режим ожидания сообщений от пользователей и других событий от платформы Telegram.

### **3.1.2. Управление состояниями диалогов**

Для обеспечения многошагового взаимодействия с пользователями в системе реализован механизм отслеживания состояний диалогов. Этот механизм позволяет системе «помнить», на каком этапе взаимодействия находится каждый пользователь, и корректно обрабатывать его сообщения в зависимости от текущего контекста.

Система состояний организована в четыре функциональные группы:

Группа состояний авторизации (AuthState) - управляет процессом получения доступа к системе:

- Ожидание ввода одноразового пароля
- Проверка корректности введенных данных
- Регистрация пользователя в системе

Группа состояний поиска по названию (SearchState) - реализует сценарий поиска предметов по текстовому запросу:

- Ожидание ввода поисковой фразы
- Ожидание выбора предмета из списка результатов

Группа состояний поиска по категориям (CategoryState) - обеспечивает навигацию по иерархической структуре категорий:

- Ожидание выбора основной категории
- Ожидание выбора подкатегории
- Ожидание выбора конкретного предмета

Группа состояний администрирования (AdminState) - управляет выполнением административных операций:

- Ожидание ввода пароля для удаления
- Ожидание ввода идентификатора пользователя для блокировки

## **3.2. Модули бота и их взаимодействие**

### **3.2.1. Модуль импорта данных (db\_importer.py)**



Данный модуль представляет собой самостоятельную программу, выполняющую функцию первичной обработки и сохранения справочной информации о предметах игры.

Основные функции модуля:

- Получение данных из внешних источников - загрузка информации из официального репозитория проекта Stalcraft Database, размещенного на платформе GitHub, или из локальной распакованной копии.
- Обработка и нормализация данных - преобразование исходных данных из формата JSON в структурированный вид, локализация названий категорий (перевод системных обозначений на русский язык), разделение категорий на основные и подкатегории.
- Сохранение в базу данных - запись обработанных данных в реляционную базу данных SQLite с использованием системы управления объектно-реляционного отображения SQLAlchemy.
- Обеспечение целостности данных - реализация механизма «вставки или обновления» (UPSERT), который предотвращает создание дубликатов записей при повторном запуске импорта.

Режимы работы модуля:

- Автоматическая загрузка - скачивание актуальной версии базы данных непосредственно из репозитория GitHub
- Локальная обработка - использование предварительно распакованной папки с данными для работы без интернет-соединения

Модуль выполняется отдельно от основного бота и не требует его остановки или перезапуска. Это позволяет обновлять справочную информацию о предметах независимо от работы системы поиска.

### **3.2.2. Модуль клиента API (bot\_api\_client.py)**

Модуль клиента обеспечивает взаимодействие с официальным программным интерфейсом игры Stalcraft для получения актуальных данных об аукционных лотах в режиме реального времени.

Основные функции модуля:

1. Аутентификация - получение и автоматическое обновление токенов доступа с использованием стандарта OAuth 2.0. Модуль самостоятельно отслеживает время жизни токена и запрашивает новый до истечения срока действия текущего.
2. Формирование запросов - подготовка HTTP-запросов к различным конечным точкам API с необходимыми параметрами (идентификатор предмета, параметры сортировки, пагинация).

3. Обработка пагинации - автоматическое получение всех страниц результатов при запросе списка лотов, так как API возвращает данные ограниченными порциями.
4. Обработка ошибок - корректная реакция на различные типы ошибок (сетевые проблемы, превышение лимитов запросов, неверные параметры) с возможностью повторных попыток.
5. Преобразование данных - конвертация полученных данных из формата JSON во внутреннюю структуру, удобную для дальнейшей обработки и отображения пользователю.

Модуль инкапсулирует все детали сетевого взаимодействия, предоставляя основному модулю бота простой высокоуровневый интерфейс для получения информации о лотах.

### **3.2.3. Основной модуль бота (main\_bot.py)**

Основной модуль представляет собой ядро системы, координирующее работу всех компонентов и обеспечивающее взаимодействие с пользователями через платформу Telegram.

Основные функции модуля:

1. Обработка входящих событий - прием и классификация сообщений, команд и нажатий кнопок от пользователей.
2. Управление диалогами - отслеживание текущего состояния каждого пользователя и маршрутизация его сообщений соответствующим обработчикам.
3. Проверка прав доступа - верификация авторизации пользователя перед выполнением защищенных операций.
4. Координация подсистем - вызов функций поиска в базе данных предметов, запросов к API аукциона, записи в журнал событий.
5. Формирование ответов - подготовка и отправка пользовательских сообщений с использованием различных типов клавиатур и форматирования.

## **3.3. Система авторизации и безопасности**

### **3.3.1. Многоуровневая система доступа**

Система реализует трехуровневую модель контроля доступа, обеспечивающую защиту функционала от несанкционированного использования.

Уровни доступа:

### Уровень 1: Неавторизованные пользователи

Пользователи, не прошедшие процедуру регистрации в системе, имеют доступ только к базовым командам:

- Команда /start - получение главного меню
- Команда /auth - инициация процесса авторизации

Все остальные запросы от неавторизованных пользователей отклоняются с сообщением о необходимости получения доступа.

### Уровень 2: Авторизованные пользователи

Пользователи, успешно прошедшие процедуру регистрации, получают полный доступ к функционалу поиска и анализа аукционной информации:

- Поиск предметов по названию
- Навигация по категориям
- Просмотр детальной информации о лотах
- Получение справки по командам

### Уровень 3: Администратор системы

Пользователь с идентификатором, указанным в конфигурации системы, получает расширенные права управления:

- Генерация одноразовых паролей для новых пользователей
- Просмотр списка активных паролей
- Удаление скомпрометированных паролей
- Просмотр списка всех авторизованных пользователей
- Блокировка доступа отдельных пользователей
- Просмотр статистики использования системы
- Получение уведомлений о критических событиях

### 3.3.2. Одноразовые пароли

Для авторизации используется система одноразовых паролей с следующими характеристиками (Таблица 10):

| Характеристика | Реализация                           | Примечание                          |
|----------------|--------------------------------------|-------------------------------------|
| Хранение       | В оперативной памяти (множество set) | Пароли не сохраняются на диск       |
| Генерация      | Алгоритм secrets.token_hex(4)        | 8-символьные шестнадцатеричные коды |

| Характеристика    | Реализация         | Примечание                          |
|-------------------|--------------------|-------------------------------------|
| Использование     | Единоразовое       | Удаление после успешной авторизации |
| Администрирование | Через админ-панель | Генерация и удаление паролей        |

*Таблица 10. Система одноразовых паролей*

Процесс авторизации пользователя (представлен на (Блок-схема 5) и на (Изобр. 10)):

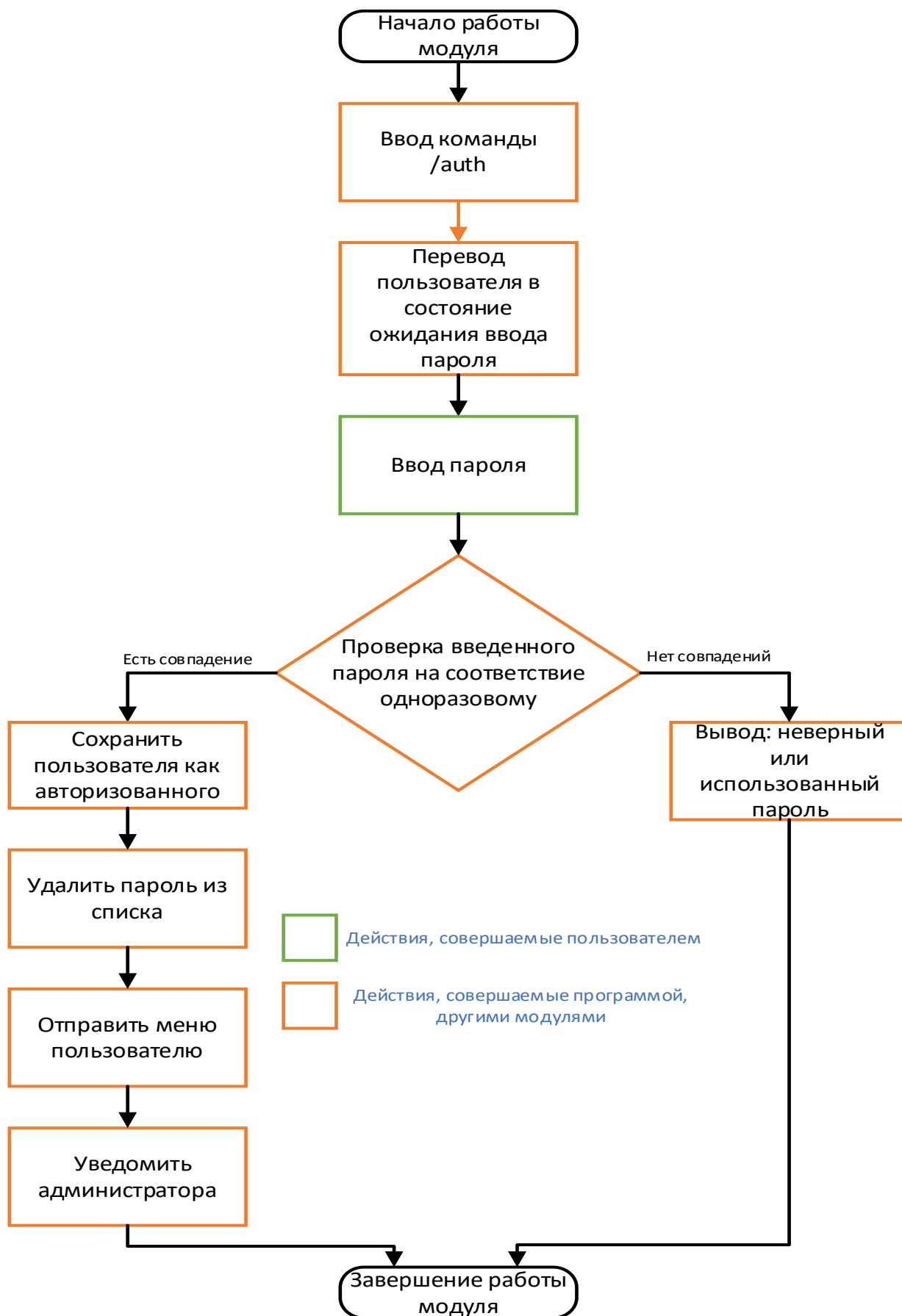
1. Инициация процесса - пользователь отправляет команду /auth боту.
2. Запрос пароля - бот переходит в состояние ожидания пароля и выводит сообщение с просьбой ввести полученный от администратора код.
3. Ввод пароля - пользователь вводит полученный одноразовый пароль.
4. Проверка корректности - система выполняет поиск введенного пароля в множестве активных паролей.
5. Обработка результата:
  - При успешной проверке:
    - Пароль удаляется из множества активных паролей
    - Данные пользователя (идентификатор, имя, имя пользователя) сохраняются в базу данных
    - Пользователь добавляется в кэш авторизованных пользователей в оперативной памяти
    - Отправляется уведомление администратору о регистрации нового пользователя
    - Пользователь получает доступ к основному меню
  - При неудачной проверке:
    - Пользователю отправляется сообщение об ошибке
    - Состояние сбрасывается
    - Предлагается повторить попытку или обратиться к администратору

### **3.3.3. Ограничение запросов (Rate Limiting)**

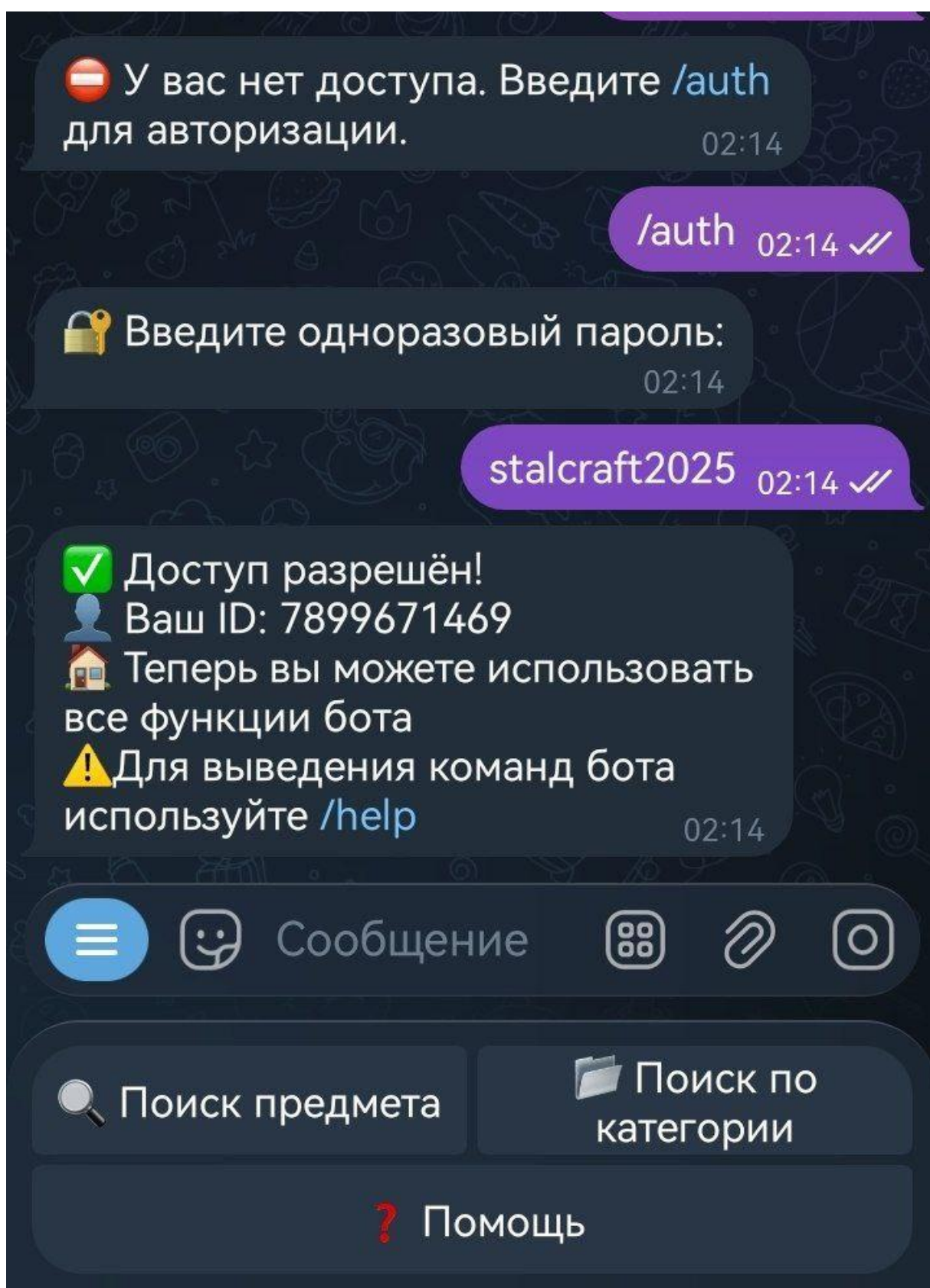
Для защиты системы от злоупотреблений и обеспечения справедливого распределения ресурсов между всеми пользователями реализован механизм ограничения частоты запросов (Rate Limiting).

Алгоритм работы:

1. Регистрация запроса - при поступлении поискового запроса от пользователя система извлекает статистику использования для данного пользователя из словаря в оперативной памяти.
2. Проверка временного окна - система вычисляет разницу между текущим временем и временем последнего запроса:
  - Если разница превышает 60 секунд, счетчик запросов сбрасывается до нуля, и временная метка обновляется
  - Если разница меньше 60 секунд, проверка продолжается
3. Проверка лимита - система сравнивает текущее значение счетчика с установленным лимитом:
  - Если счетчик достиг или превысил лимит, запрос блокируется
  - Пользователю отправляется сообщение с указанием времени до разблокировки
  - В журнал событий записывается факт превышения лимита
4. Разрешение запроса - если лимит не превышен:
  - Счетчик увеличивается на единицу
  - Запрос передается на обработку
  - Временная метка последнего запроса обновляется



Блок-схема 5. Процесс авторизации пользователя



Изображение 10. Авторизация от лица пользователя

Преимущества реализации:

- Защита от спама - предотвращение массовых автоматических запросов
- Справедливое распределение - обеспечение равного доступа к ресурсам для всех пользователей
- Снижение нагрузки - ограничение количества обращений к базе данных и внешнему API
- Информирование - пользователи получают четкую информацию о времени ожидания

### 3.3.4. Защита административных функций

Доступ к функциям управления системой строго ограничен и контролируется на нескольких уровнях.

Механизмы защиты:

1. Проверка идентификатора - при каждой попытке доступа к административным функциям система сверяет идентификатор пользователя с предустановленным значением администратора из конфигурационного файла.
2. Изоляция команд - команда /admin и все callback-запросы от административной панели обрабатываются отдельными обработчиками, которые выполняют проверку прав до начала выполнения любой логики.
3. Логирование действий - все административные операции (генерация паролей, блокировка пользователей, удаление паролей) фиксируются в журнале событий с указанием времени, типа действия и целевого объекта.
4. Многошаговые операции - критические операции (удаление паролей, блокировка пользователей) требуют дополнительного подтверждения через ввод данных, что снижает риск случайного выполнения.

## 3.4. Пользовательский интерфейс и навигация

### 3.4.1. Главное меню и команды

Бот предоставляет интуитивно понятный интерфейс с использованием клавиатур двух типов:

- **Reply-клавиатуры** - для основных действий
- **Inline-клавиатуры** - для контекстных действий и админ-панели

Основные команды бота представлены в (Таблица 11):

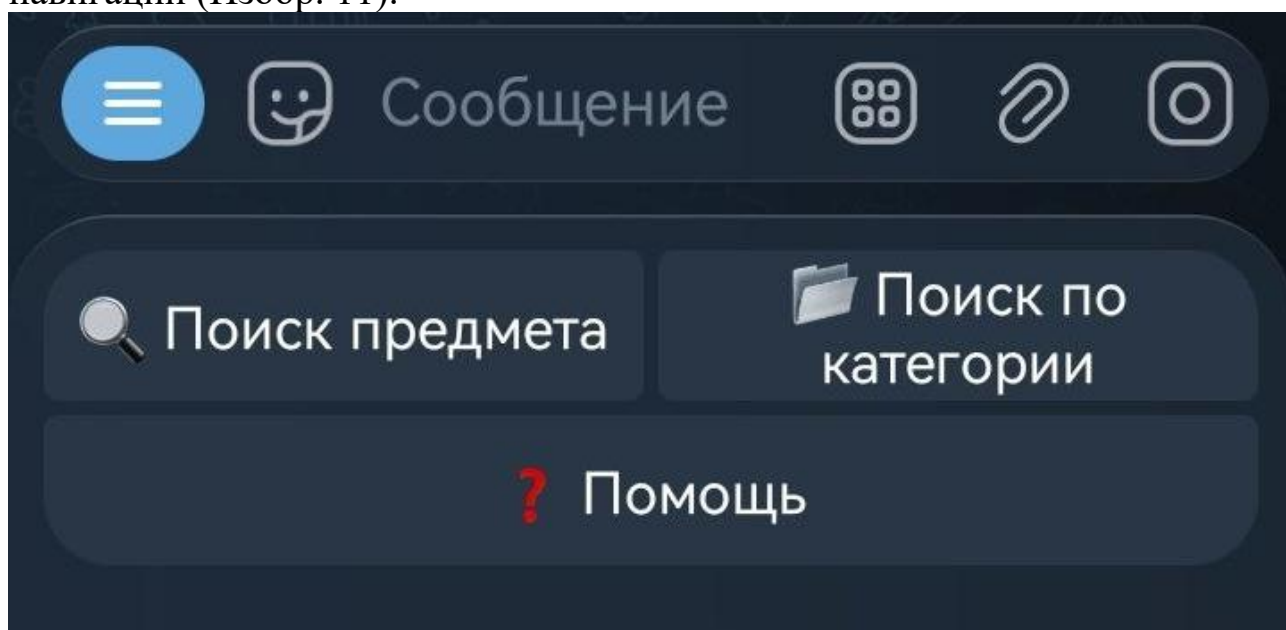


| Команда | Назначение                | Доступ               |
|---------|---------------------------|----------------------|
| /start  | Главное меню              | Все пользователи     |
| /auth   | Авторизация               | Неавторизованные     |
| /help   | Справка                   | Авторизованные       |
| /info   | Информация о пользователе | Авторизованные       |
| /admin  | Админ-панель              | Только администратор |

Таблица 11. Основные команды бота

### 3.4.2. Дизайн интерфейса

Интерфейс бота построен на принципах минимализма и удобства навигации (Изобр. 11).



Изображение 11. Главное меню бота

### 3.4.3. Система навигации

Система поддерживает несколько навигационных путей для выполнения различных задач, каждый из которых оптимизирован под конкретный сценарий использования.

Сценарий 1: Поиск по названию

Последовательность шагов:

1. Пользователь нажимает кнопку «Поиск предмета» в главном меню
2. Система переходит в состояние ожидания поискового запроса

3. Отображается клавиатура с кнопкой отмены
4. Пользователь вводит часть названия предмета (например, «ак-74»)
5. Система выполняет поиск в базе данных с использованием нечеткого сопоставления
6. В зависимости от количества найденных результатов:
  - 0 результатов: вывод сообщения «Ничего не найдено» с рекомендацией уточнить запрос
  - 1 результат: автоматический переход к отображению детальной карточки предмета
  - 2 и более результатов: вывод списка найденных предметов с клавиатурой для выбора
7. При множественных результатах пользователь нажимает кнопку с названием нужного предмета
8. Система выполняет проверку корректности выбора
9. Отображается детальная карточка предмета с информацией о лотах
10. Возврат в главное меню

## Сценарий 2: Поиск по категориям

### Последовательность шагов:

1. Пользователь нажимает кнопку «Поиск по категории» в главном меню
2. Система выполняет запрос к базе данных для получения списка основных категорий
3. Отображается клавиатура со списком категорий (Оружие, Броня, Артефакты и т.д.)
4. Пользователь выбирает основную категорию (например, «Броня») - (Изобр. 12)
5. Система выполняет запрос для получения списка подкатегорий выбранной категории
6. Отображается клавиатура со списком подкатегорий (Пистолеты, Штурмовые винтовки и т.д.)
7. Пользователь выбирает подкатегию (например, «Броня/Комбинированные») - (Изобр. 13)
8. Система выполняет запрос для получения списка предметов в выбранной подкатегории
9. Отображается клавиатура со списком предметов
10. Пользователь выбирает конкретный предмет - (Изобр. 14)
11. Отображается детальная карточка предмета
12. Возврат в главное меню

### Сценарий 3: Навигация с возвратом

На всех уровнях навигации, кроме главного меню, доступна кнопка «Назад», которая позволяет:

- Очистить текущее состояние
- Возвратиться в главное меню
- Скрыть текущую клавиатуру
- Вывести сообщение об отмене

1. Главное меню → Поиск → Результаты → Детальная информация
2. Главное меню → Категории → Подкатегории → Предметы → Детальная информация
3. Админ-панель → Разделы управления → Возврат в меню

## 3.5. Поиск предметов и вывод информации

### 3.5.1. Два метода поиска

Бот реализует две стратегии поиска, соответствующие описанным в Главе 1:

#### А. Поиск по названию

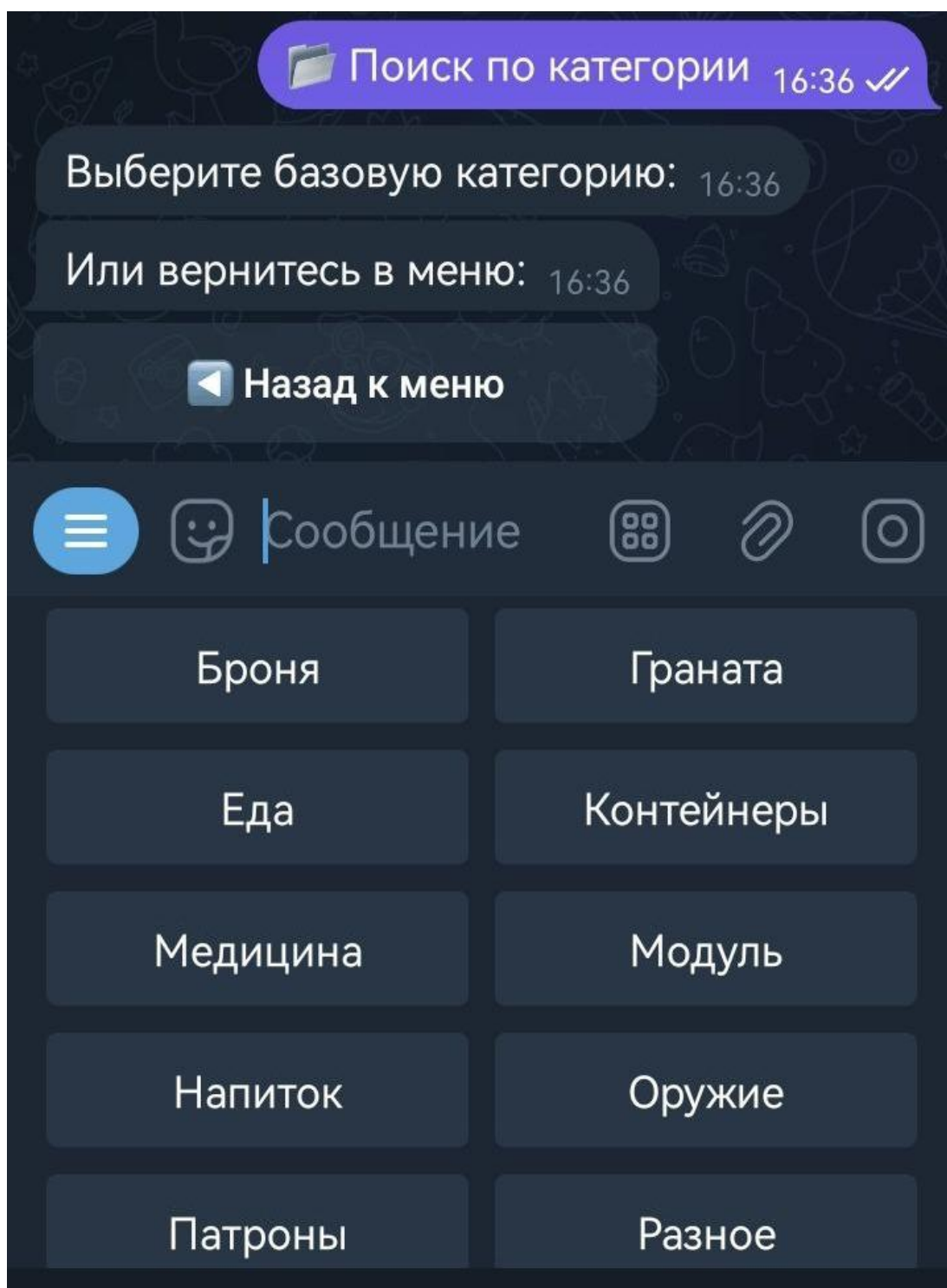
Пошаговый алгоритм:

1. Пользователь выбирает "Поиск предмета"
2. Вводит часть названия
3. Система выполняет нечеткий поиск
4. В зависимости от результатов:
  - 0 результатов → сообщение "Ничего не найдено"
  - 1 результат → автоматический переход к карточке
  - 2+ результата → выбор из списка

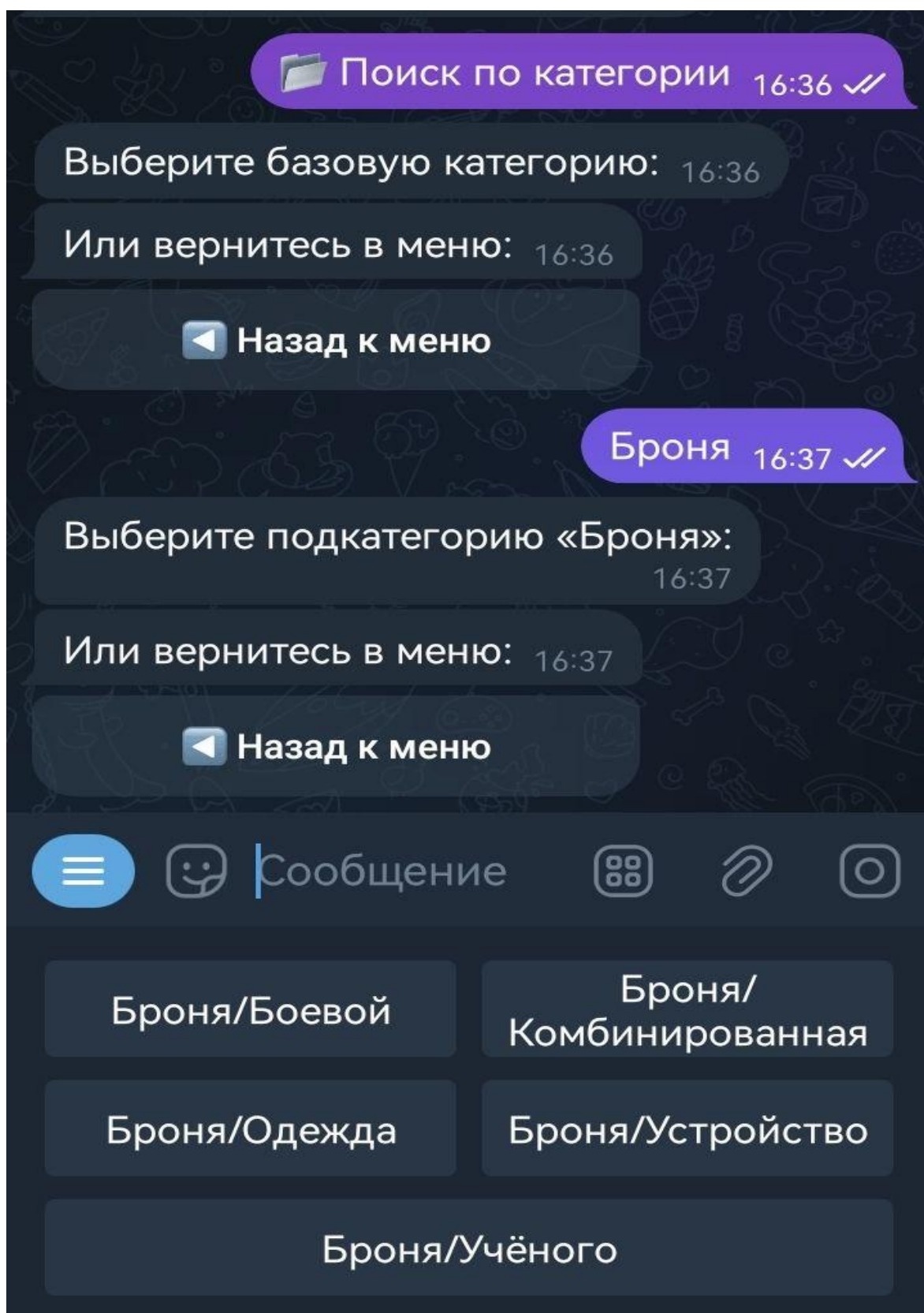
#### Б. Поиск по категориям

Иерархическая навигация по категориям:

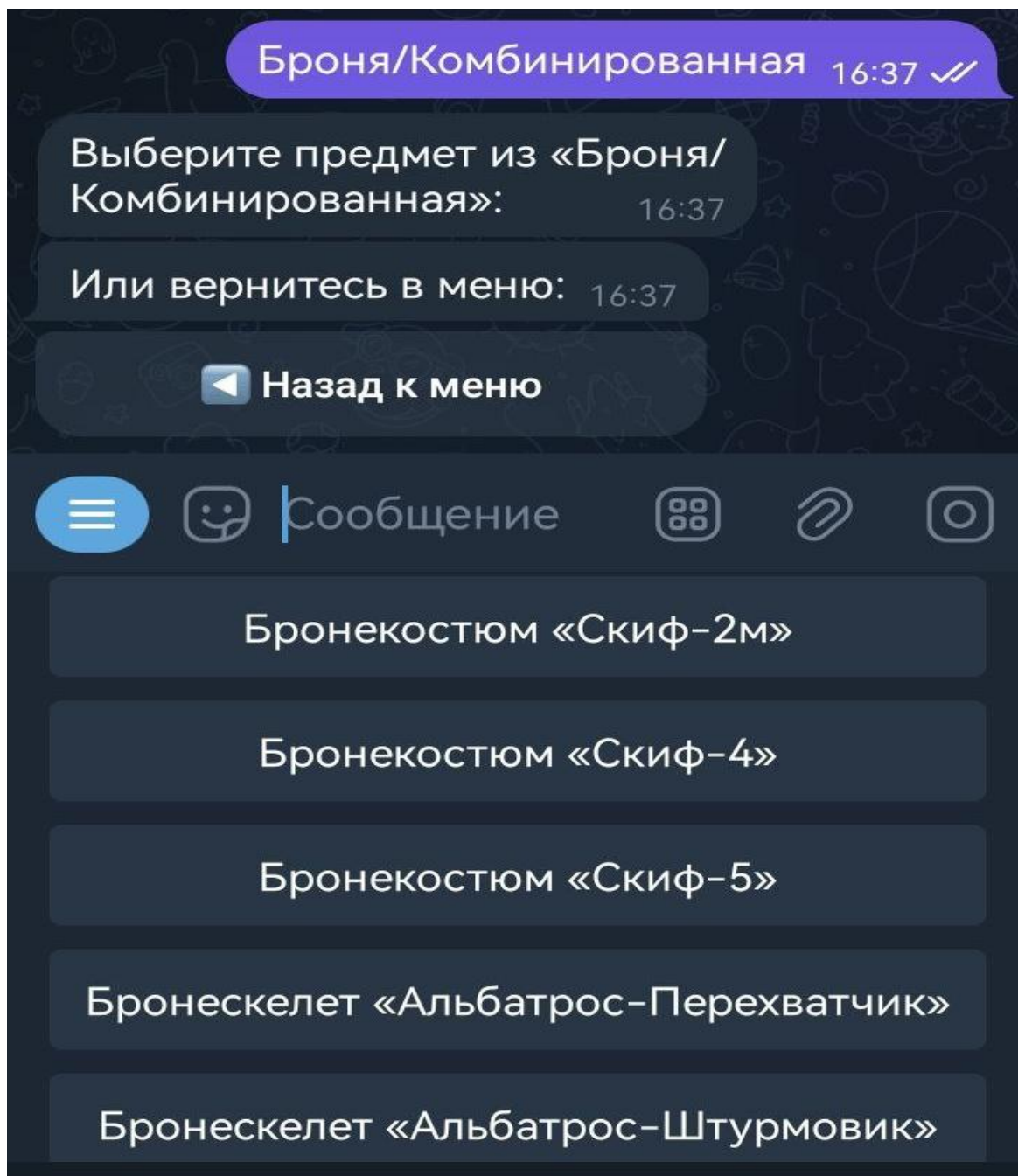
1. Выбор базовой категории (Изобр. 12)
2. Выбор подкатегории (Изобр. 13)
3. Выбор предмета из списка (Изобр. 14)



Изображение 12. Выбор базовой категории



Изображение 13. Выбор подкатегории



*Изображение 14. Выбор предмета из списка*

### **3.5.2. Детальная карточка предмета**

При выборе предмета из списка результатов система формирует подробную карточку с информацией о предмете и доступных аукционных лотах.

Структура карточки:

Блок 1: Основная информация о предмете

Содержит справочные данные из базы предметов:

- Название предмета - полное наименование на русском языке с эмодзи для визуального выделения
- Категория - иерархический путь в формате «Основная категория/Подкатегория»

## Блок 2: Статистика аукциона

Предоставляет сводные данные для оценки рыночной ситуации:

- Общее количество активных лотов - показывает ликвидность предмета (насколько активно он продается)
- Лучшая цена выкупа - минимальная цена мгновенной покупки среди всех активных лотов
- Средняя цена топ-5 - среднее арифметическое цен пяти самых дешевых лотов для понимания типичного уровня цен

Форматирование числовых значений:

- Цены выводятся с разделителем тысяч (запятая) для улучшения читаемости больших чисел
- Пример: 15000 Р отображается как «15,000 Р»

## Блок 3: Список топ-5 самых дешевых лотов

Детализированная информация о пяти наиболее выгодных предложениях:

Для каждого лота отображается:

- Порядковый номер - для удобства ссылки на конкретный лот
- Количество единиц - сколько предметов содержится в лоте
- Текущая ставка (если есть) - цена, до которой поднялись торги
- Цена выкупа (если есть) - цена мгновенной покупки без участия в торгах
- Время окончания - дата и время завершения лота в формате «ДД.ММ ЧЧ:ММ»

Обработка отсутствующих данных:

- Если для лота доступна только одна цена (либо ставка, либо выкуп), отображается только она
- Если обе цены отсутствуют, выводится «Цена: -»
- Если время окончания не указано, выводится символ «-»

## Блок 4: Акцент на лучший лот

Выделенный блок с наиболее выгодным предложением:

- Визуальные маркеры - эмодзи трофея и заголовков заглавными буквами
- Полная информация - аналогична блоку 3, но для лучшего лота
- Призыв к действию - текст «Успейте купить по лучшей цене!» для стимулирования быстрого принятия решения

Алгоритм формирования:

1. Получение справочной информации - запрос к базе данных предметов по идентификатору для получения названия и категории.
2. Запрос данных об лотах - обращение к официальному API игры для получения списка всех активных лотов данного предмета с сортировкой по цене выкупа по возрастанию.

3. Обработка и фильтрация - исключение лотов без цены (оба поля null), извлечение необходимых полей (количество, текущая цена, цена выкупа, время окончания).
4. Сортировка - упорядочивание лотов по возрастанию цены для выявления наиболее выгодных предложений.
5. Расчет статистики - вычисление минимальной цены и среднего значения для топ-5.
6. Форматирование - преобразование временных меток из формата ISO 8601 в человекочитаемый вид, добавление разделителей тысяч к ценам.
7. Отправка сообщений - последовательная отправка блоков карточки пользователю с разделением на несколько сообщений для удобства чтения.

Пример отображения карточки предмета представлен на (Изобр. 15).

### **3.6. Административные функции**

#### **3.6.1. Структура админ-панели**

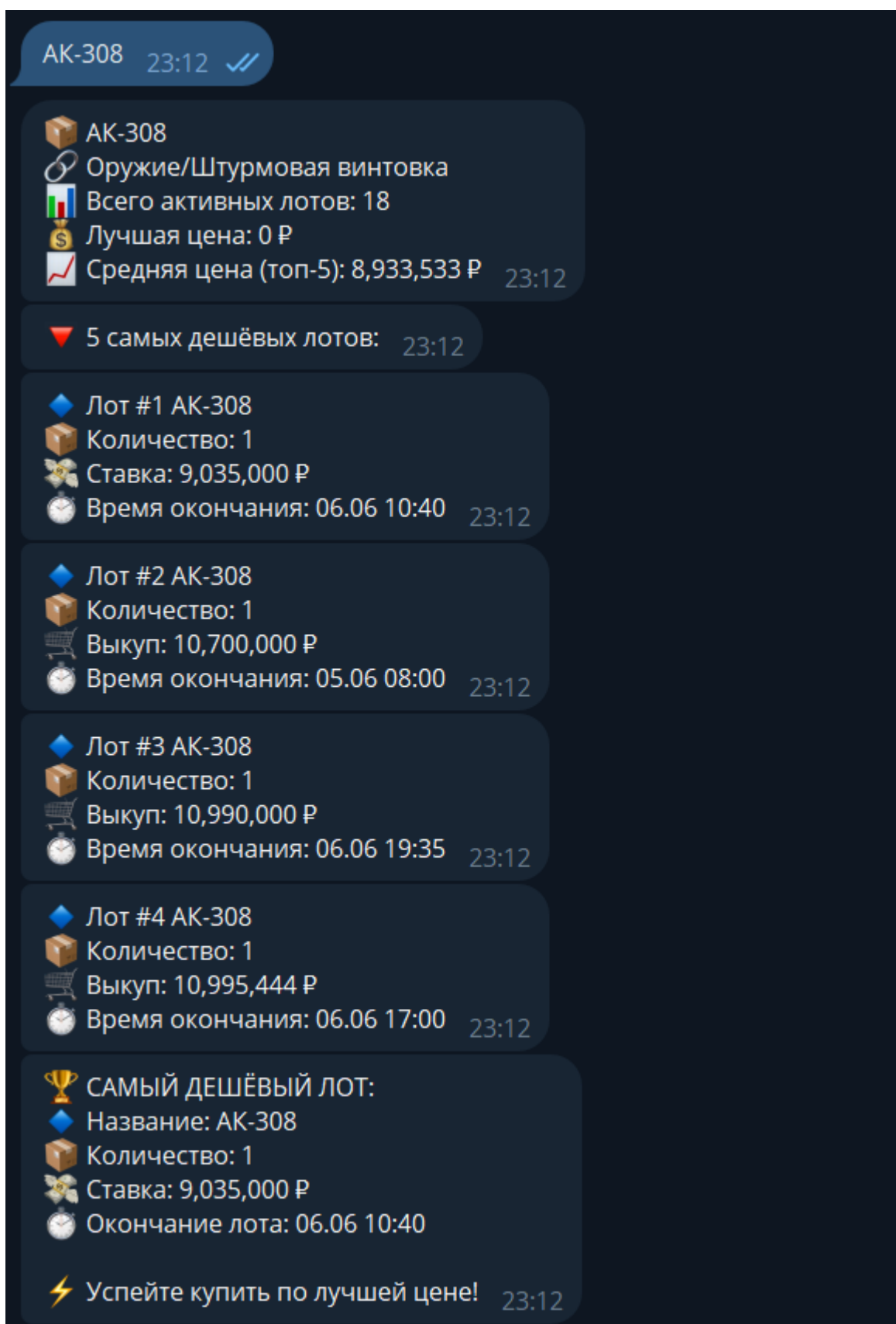
Административный интерфейс предоставляет полный набор инструментов для управления системой, контроля доступа и мониторинга использования.

Административная панель (Таблица 12) предоставляет полный контроль над системой через inline-клавиатуру с шестью основными функциями (Изобр. 16).

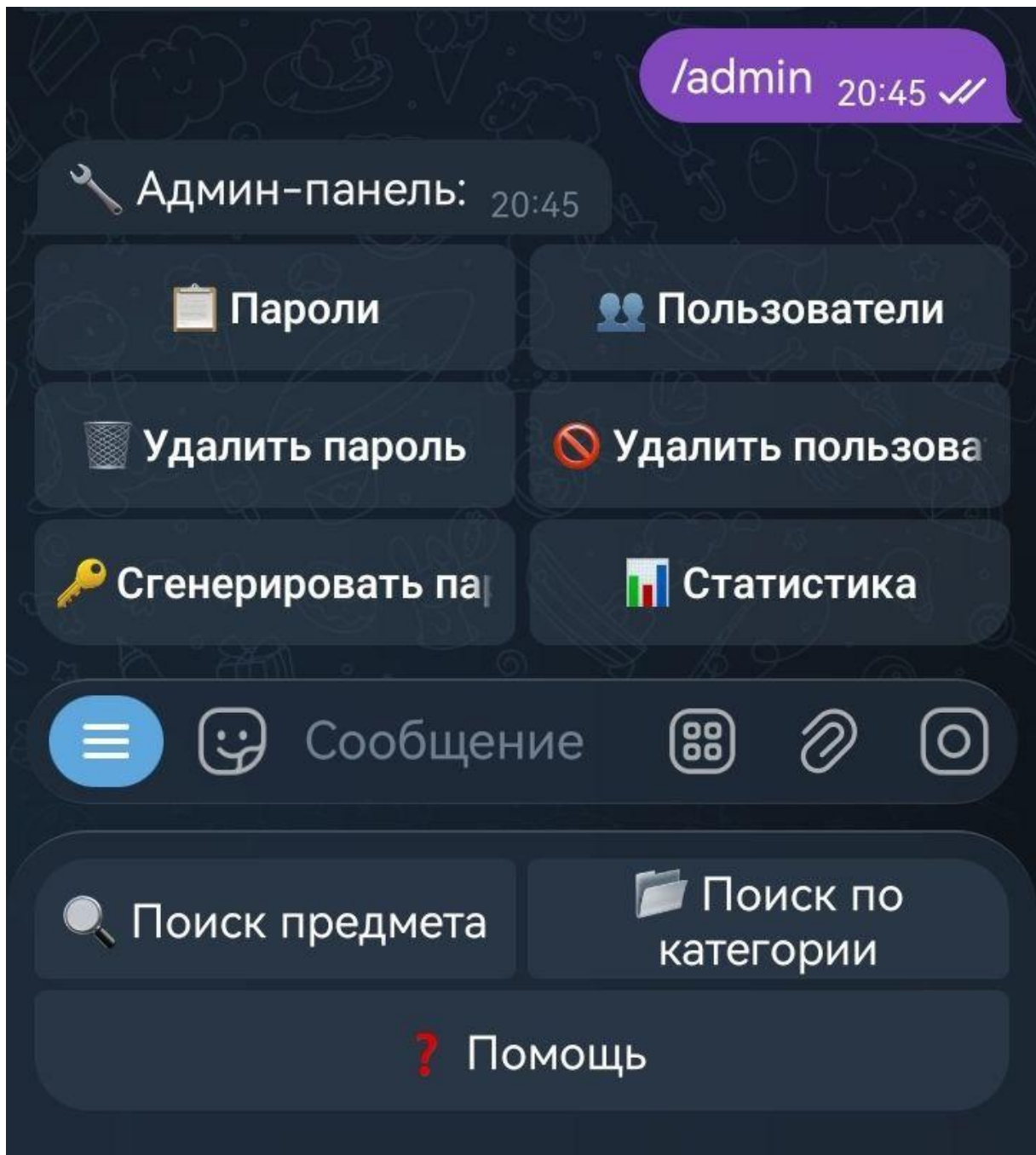


| Функция              | Описание                              | Метод доступа         |
|----------------------|---------------------------------------|-----------------------|
| Пароли               | Просмотр активных одноразовых паролей | Inline-кнопка         |
| Пользователи         | Список авторизованных пользователей   | Inline-кнопка         |
| Удалить пароль       | Удаление конкретного пароля           | Многошаговая операция |
| Удалить пользователя | Отзыв доступа у пользователя          | Многошаговая операция |
| Сгенерировать пароль | Создание нового одноразового пароля   | Inline-кнопка         |
| Статистика           | Просмотр статистики использования     | Inline-кнопка         |

*Таблица 12. Панель функций администратора*



Изображение 15. Карточка предмета с аукционной информацией



Изображение 16. Административная панель бота

### 3.6.2. Функционал администратора

Функция 1: Просмотр активных паролей

Назначение: Позволяет администратору контролировать количество и содержание активных одноразовых паролей в системе.

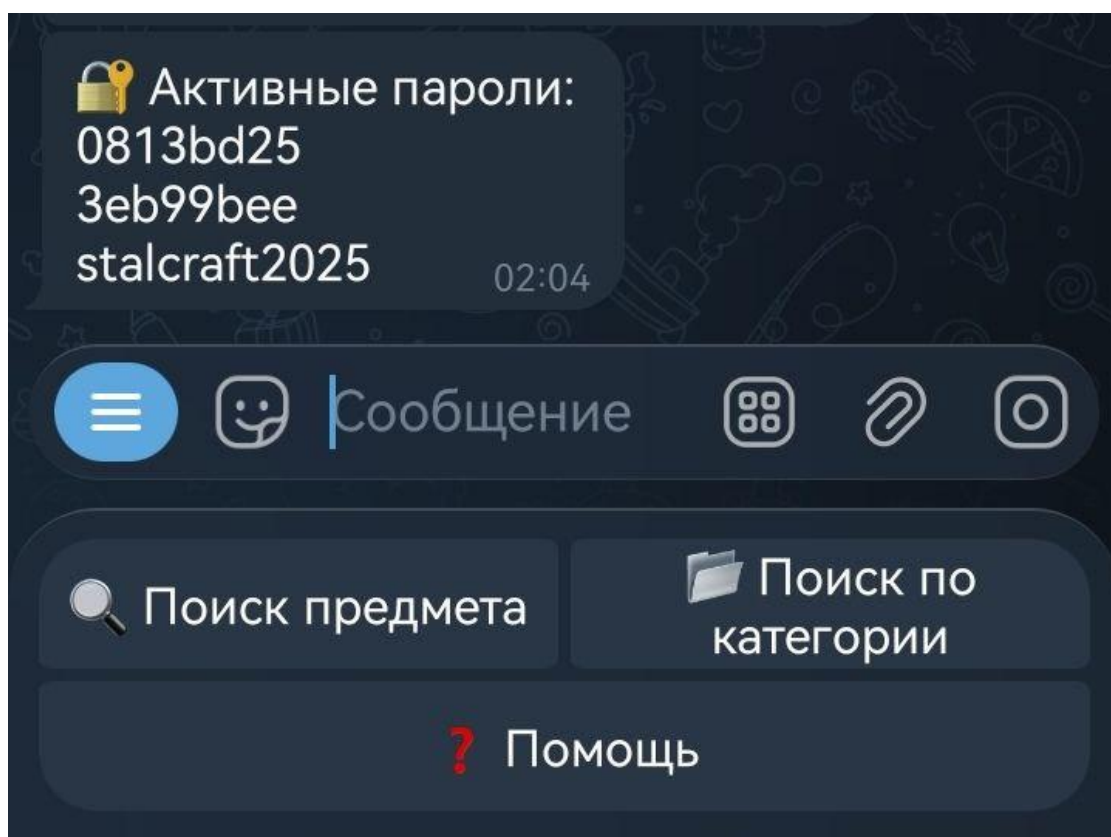
Процесс выполнения:

1. Администратор нажимает кнопку «Пароли»
2. Система проверяет множество активных паролей в оперативной памяти
3. Если паролей нет - выводится сообщение «Нет активных паролей»
4. Если пароли есть - формируется список с разделением строк

5. Список отправляется администратору в виде текстового сообщения (Изобр. 17)

Использование:

- Контроль безопасности - проверка, что паролей не слишком много
- Выявление «зависших» паролей, которые долго не используются
- Подготовка к выдаче новых паролей



*Изображение 17. Просмотр активных паролей*

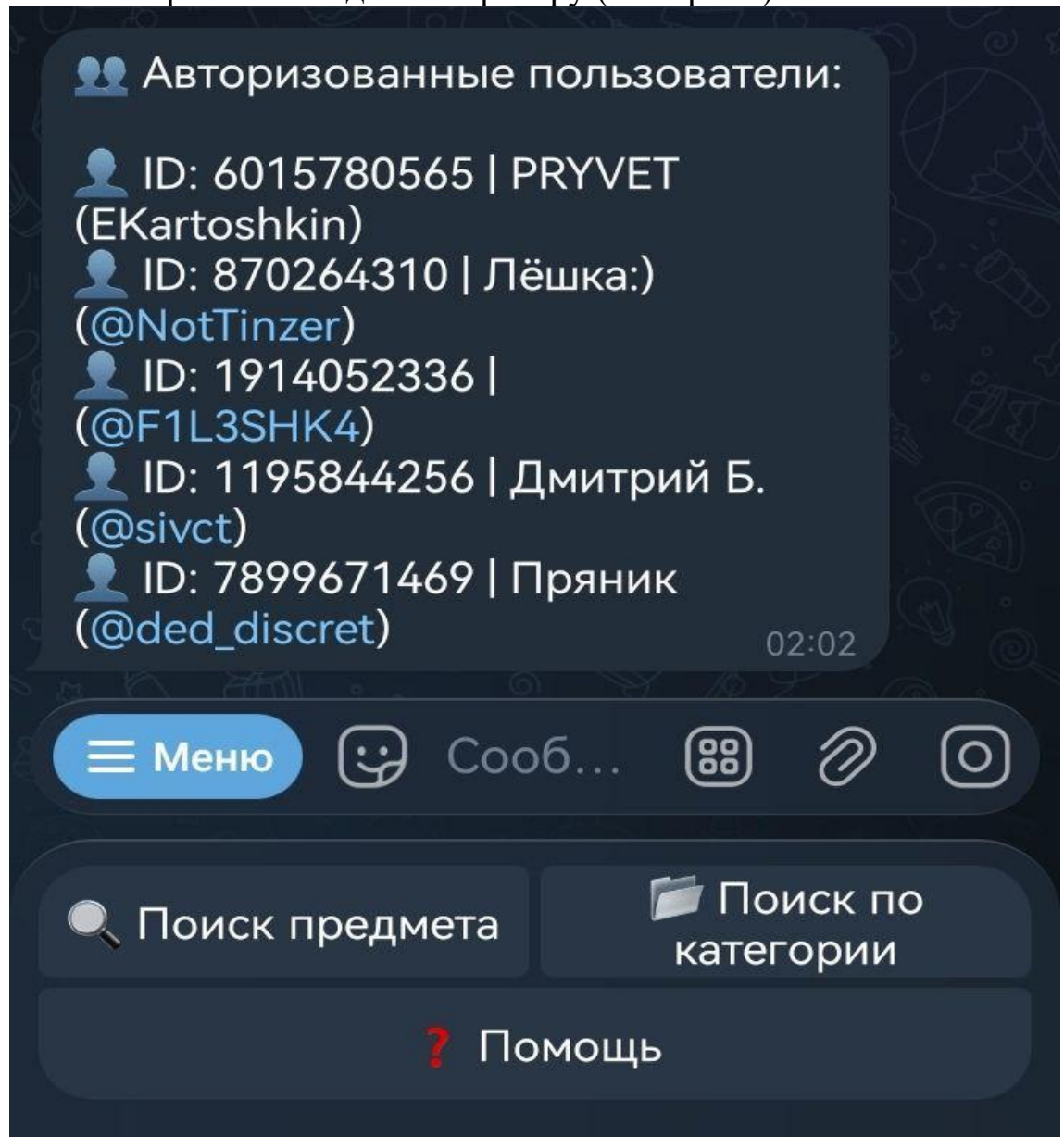
## **Функция 2: Просмотр авторизованных пользователей**

Назначение: Отображение полного списка пользователей, прошедших авторизацию и получивших доступ к системе.

Процесс выполнения:

1. Администратор нажимает кнопку «Пользователи»
2. Система выполняет запрос к базе данных пользователей с сортировкой по дате авторизации (новые сверху)
3. Для каждого пользователя извлекаются:
  - Идентификатор Telegram (user\_id)
  - Имя пользователя (username) с символом @

- Имя (first\_name)
  - Фамилия (last\_name)
4. Формируется читаемый список с форматированием
  5. Список отправляется администратору (Изобр. 18)



Изображение 18. Просмотр авторизованных пользователей

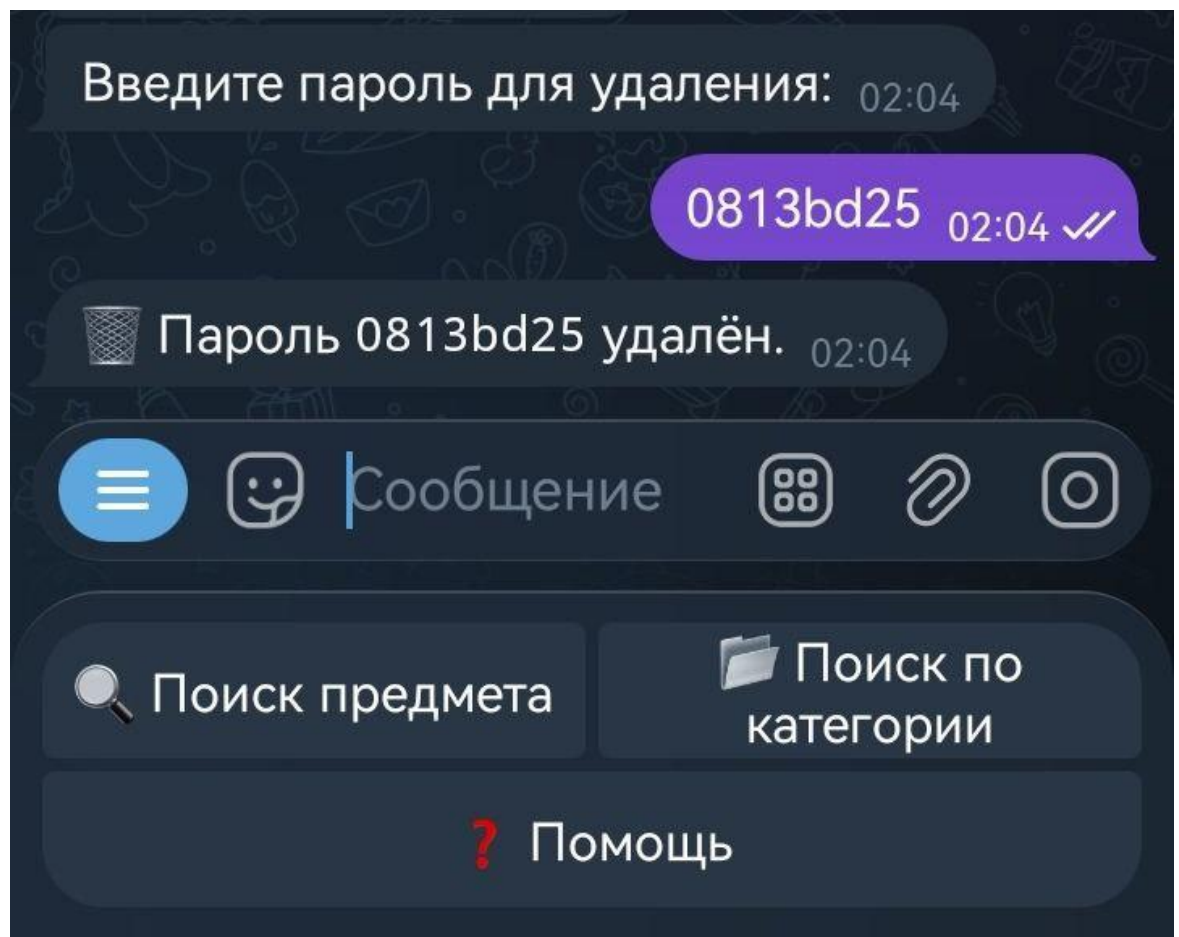
### Функция 3: Удаление пароля

Назначение: Принудительное удаление конкретного одноразового пароля из системы при его компрометации или неактуальности.

Процесс выполнения (многошаговая операция):

1. Администратор нажимает кнопку «Удалить пароль»

2. Система переходит в состояние ожидания ввода пароля
3. Отображается сообщение с просьбой ввести пароль для удаления и клавиатура с кнопкой отмены
4. Администратор вводит пароль, который необходимо удалить
5. Система проверяет наличие пароля во множестве активных паролей
6. Если пароль найден:
  - Пароль удаляется из множества
  - Отправляется сообщение об успешном удалении (Изобр. 19)
  - Запись о действии сохраняется в журнале
7. Если пароль не найден:
  - Отправляется сообщение об ошибке «Пароль не найден»
8. Состояние сбрасывается, возврат в главное меню



*Изображение 19. Удаление пароля*

#### **Функция 4: Блокировка пользователя**

Назначение: Отзыв доступа у пользователя, нарушающего правила использования системы.

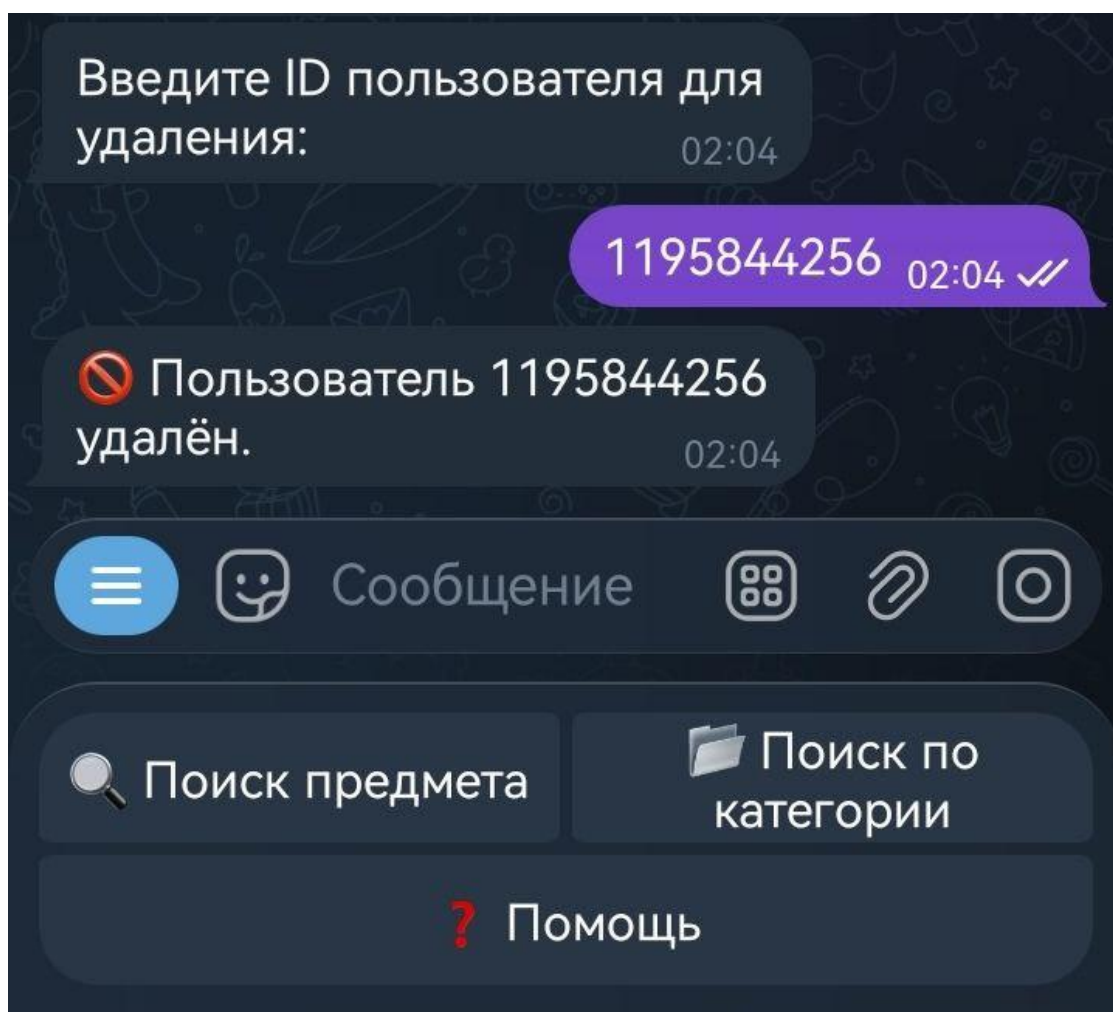
Процесс выполнения (многошаговая операция):

1. Администратор нажимает кнопку «Удалить пользователя»
2. Система переходит в состояние ожидания ввода идентификатора
3. Отображается сообщение с просьбой ввести ID пользователя и клавиатура отмены
4. Администратор вводит числовой идентификатор пользователя
5. Система проверяет наличие пользователя в словаре авторизованных пользователей
6. Если пользователь найден:
  - Выполняется обновление записи в базе данных (установка флага `is_active = 0`)
  - Пользователь удаляется из кэша в оперативной памяти
  - Отправляется сообщение об успешной блокировке (Изобр. 20)
  - Запись о действии сохраняется в журнале
7. Если пользователь не найден:
  - Отправляется сообщение «Пользователь не найден»
8. Состояние сбрасывается

Тип блокировки: «Мягкое» удаление - запись пользователя сохраняется в базе данных с флагом неактивности, что позволяет:

- Сохранить историю действий для аудита
- Восстановить доступ при необходимости (сброс флага)
- Анализировать поведение заблокированных пользователей





*Изображение 20. Удаление пользователя*

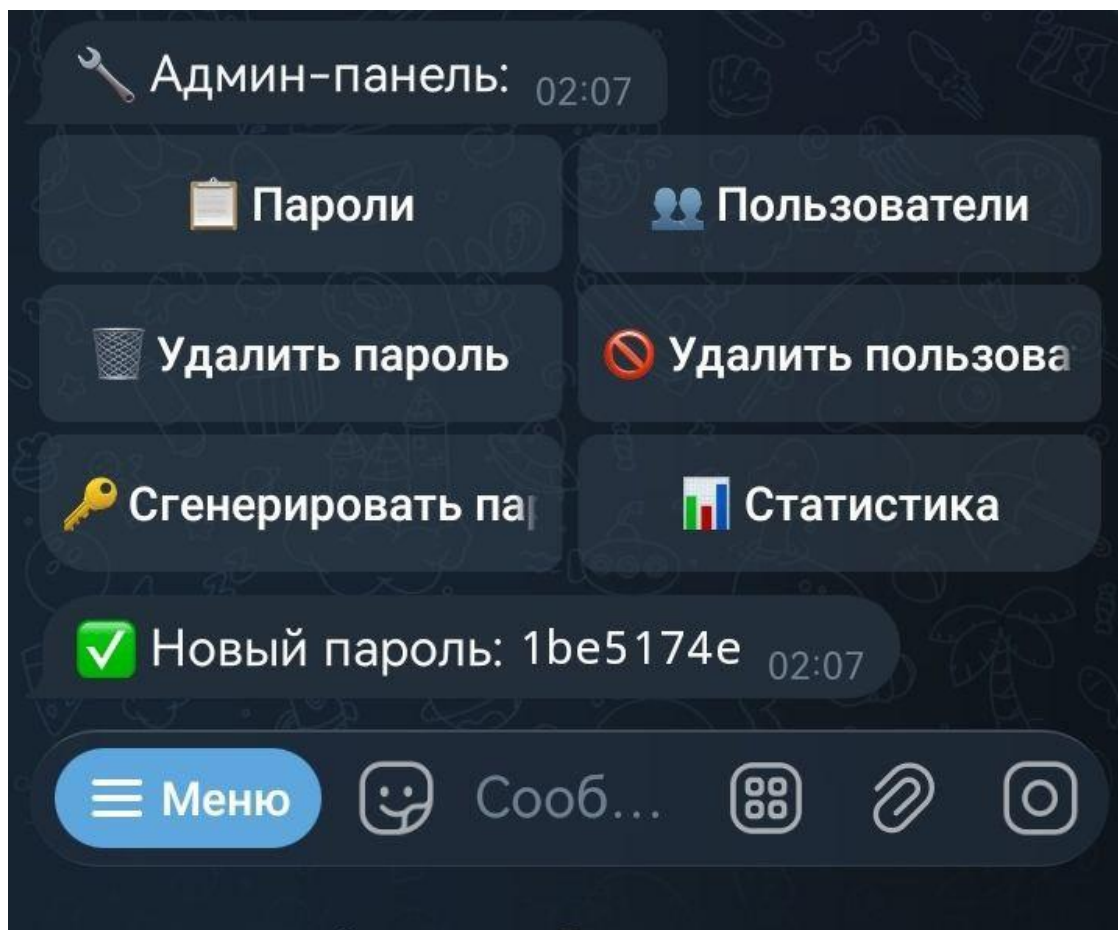
### **Функция 5: Генерация нового пароля**

Назначение: Автоматическое создание криптографически стойкого одноразового пароля для регистрации нового пользователя.

Процесс выполнения:

1. Администратор нажимает кнопку «Сгенерировать пароль»
2. Система вызывает функцию генерации случайного значения с использованием модуля secrets
3. Генерируется 8-символьная шестнадцатеричная строка
4. Новый пароль добавляется во множество активных паролей
5. Пароль немедленно отображается администратору в формате Markdown для удобного копирования (Изобр. 21)
6. Запись о генерации пароля сохраняется в журнале событий





Изображение 21. Генерация нового пароля

## Функция 6: Просмотр статистики

Назначение: Предоставление администратору сводной информации о состоянии и использовании системы.

Процесс выполнения:

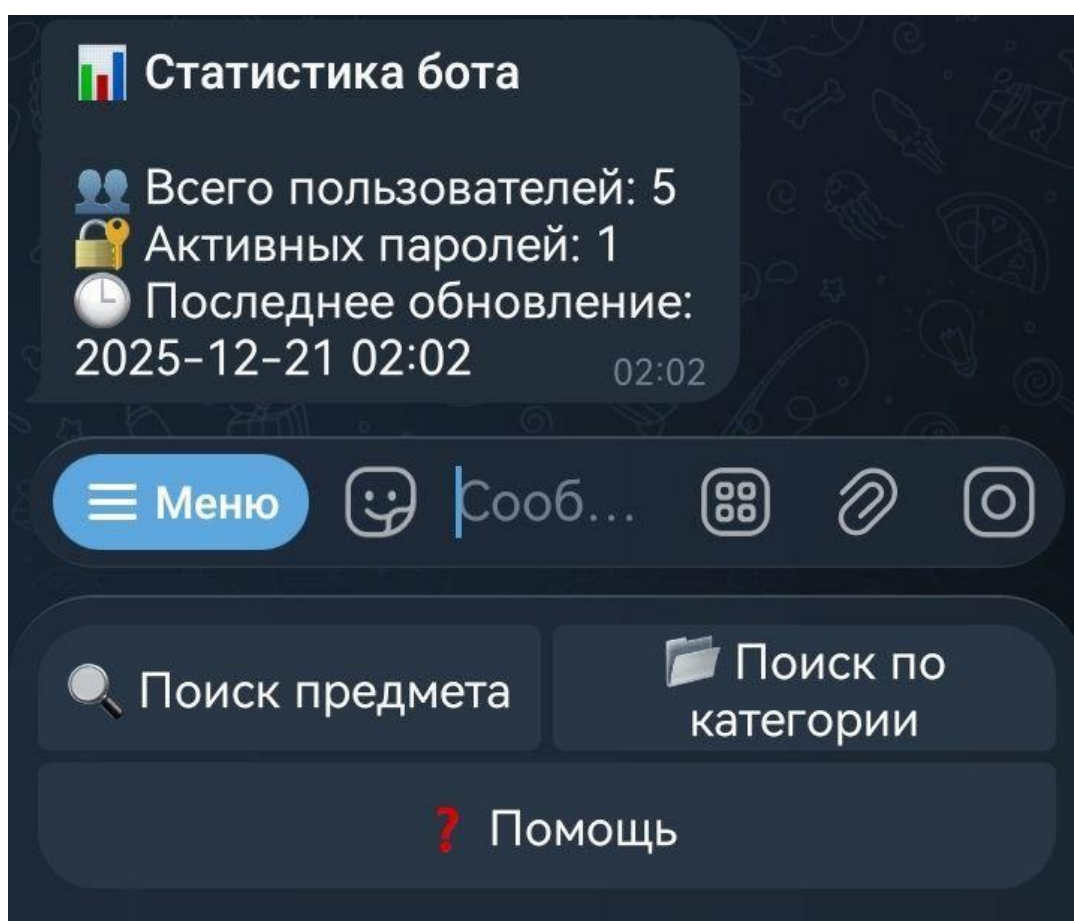
1. Администратор нажимает кнопку «Статистика»
2. Система выполняет запрос к базе данных пользователей для подсчета активных ( $is\_active = 1$ )
3. Получается количество активных паролей из множества в оперативной памяти
4. Формируется текущая дата и время
5. Отправляется форматированное сообщение со статистикой (Изобр. 22)

Показатели:

- Пользователей - общее количество авторизованных пользователей с активным статусом
- Паролей - текущее количество неиспользованных одноразовых паролей
- Время - момент формирования статистики для оценки актуальности

Использование:

- Мониторинг нагрузки на систему
- Выявление аномалий (резкий рост пользователей, накопление паролей)
- Планирование ресурсов



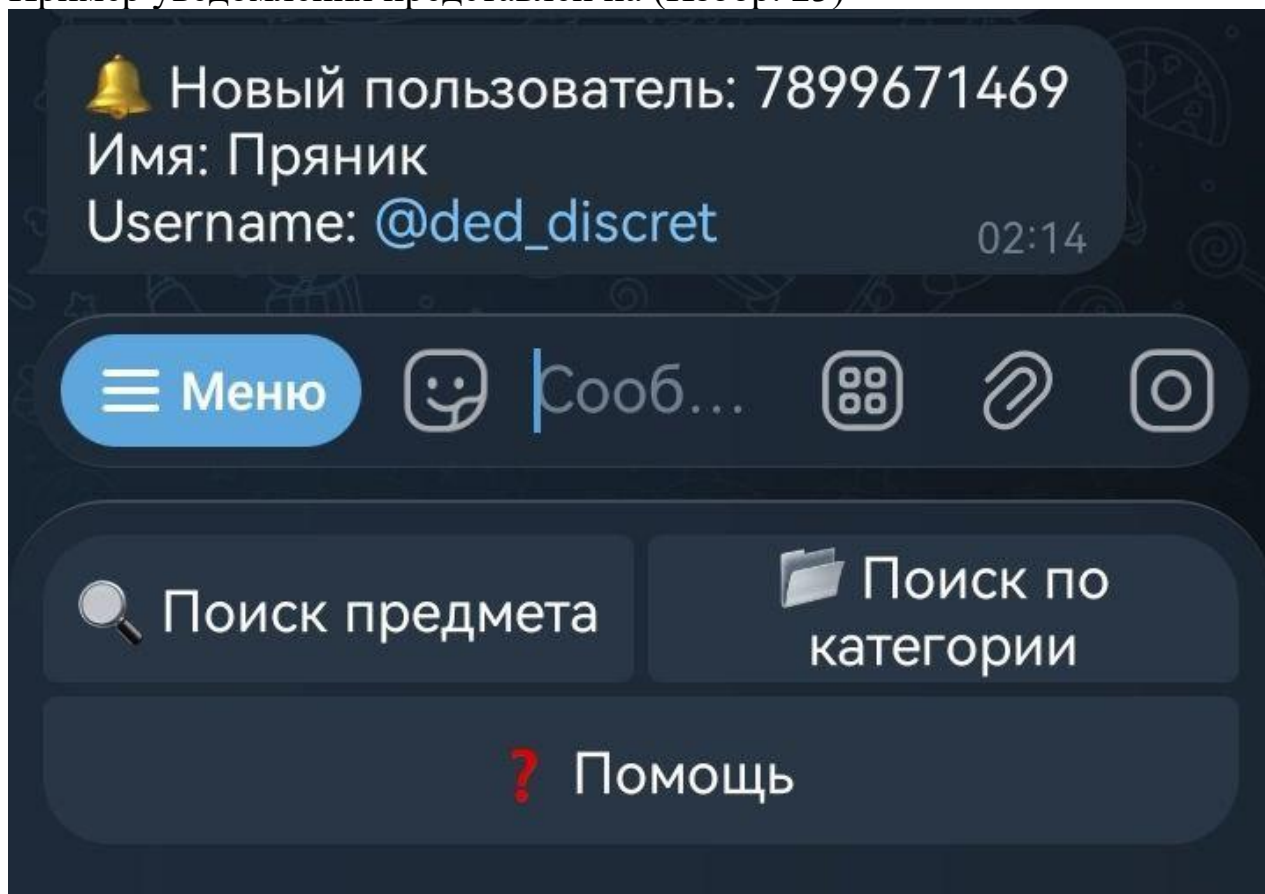
Изображение 22. Просмотр статистики

### 3.6.3. Система уведомлений

Администратор получает автоматические уведомления о ключевых событиях:

- Новая авторизация пользователя
- Критические ошибки в работе системы

Пример уведомления представлен на (Изобр. 23)



Изображение 23. Уведомление администратора

## 3.7. Обработка ошибок и логирование

### 3.7.1. Многоуровневая система обработки исключений

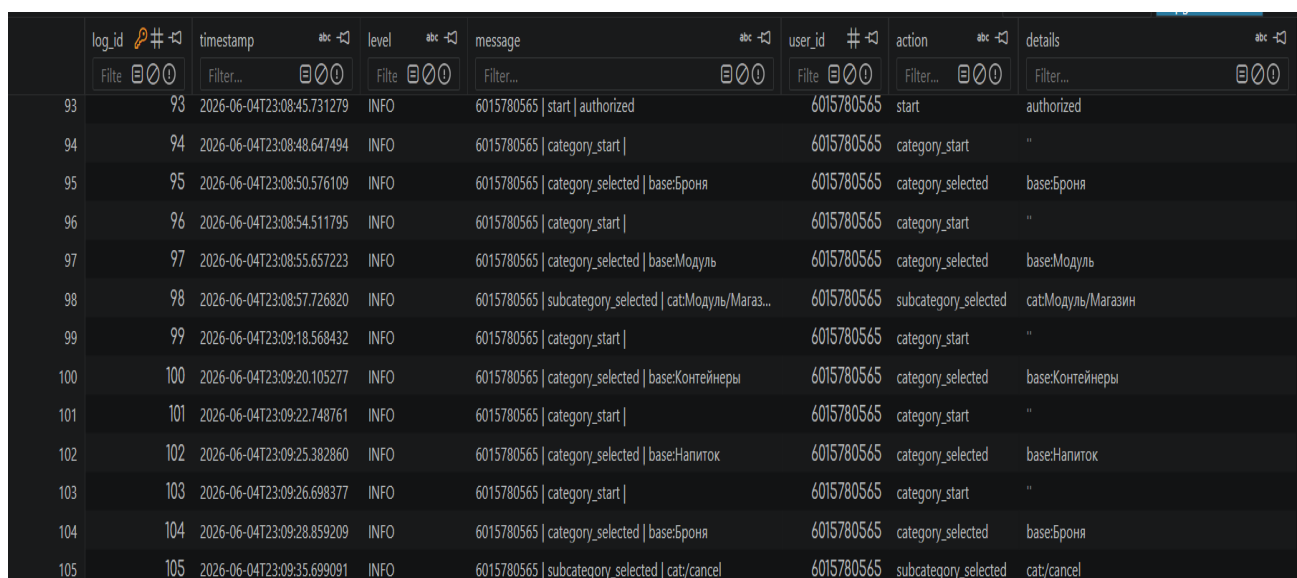
Система предусматривает обработку различных типов ошибок (Таблица 13)

| Тип ошибки          | Метод обработки                  | Действие пользователя                    |
|---------------------|----------------------------------|------------------------------------------|
| Сетевые проблемы    | Повторные попытки, кэширование   | Информирование о временной недоступности |
| Ошибки парсинга     | Резервные алгоритмы, логирование | Предложение повторить позже              |
| Некорректный ввод   | Валидация, подсказки             | Уточняющие сообщения                     |
| Ограничение доступа | Проверка прав, информирование    | Сообщение о необходимости авторизации    |

Таблица 13. Обработка ошибок

### 3.7.2. Система логирования

Реализовано логирование всех действий в системе. Все записи сохраняются в отдельной базе данных logs.db с помощью SQLite для долгосрочного хранения и последующего анализа (Изобр. 24).



| log_id | timestamp                  | level | message                                                 | user_id    | action               | details            |
|--------|----------------------------|-------|---------------------------------------------------------|------------|----------------------|--------------------|
| 93     | 2026-06-04T23:08:45.731279 | INFO  | 6015780565   start   authorized                         | 6015780565 | start                | authorized         |
| 94     | 2026-06-04T23:08:48.647494 | INFO  | 6015780565   category_start                             | 6015780565 | category_start       | "                  |
| 95     | 2026-06-04T23:08:50.576109 | INFO  | 6015780565   category_selected   base:Броня             | 6015780565 | category_selected    | base:Броня         |
| 96     | 2026-06-04T23:08:54.511795 | INFO  | 6015780565   category_start                             | 6015780565 | category_start       | "                  |
| 97     | 2026-06-04T23:08:55.657223 | INFO  | 6015780565   category_selected   base:Модуль            | 6015780565 | category_selected    | base:Модуль        |
| 98     | 2026-06-04T23:08:57.726820 | INFO  | 6015780565   subcategory_selected   cat:Модуль/Магаз... | 6015780565 | subcategory_selected | cat:Модуль/Магазин |
| 99     | 2026-06-04T23:09:18.568432 | INFO  | 6015780565   category_start                             | 6015780565 | category_start       | "                  |
| 100    | 2026-06-04T23:09:20.105277 | INFO  | 6015780565   category_selected   base:Контейнеры        | 6015780565 | category_selected    | base:Контейнеры    |
| 101    | 2026-06-04T23:09:22.748761 | INFO  | 6015780565   category_start                             | 6015780565 | category_start       | "                  |
| 102    | 2026-06-04T23:09:25.382860 | INFO  | 6015780565   category_selected   base:Напиток           | 6015780565 | category_selected    | base:Напиток       |
| 103    | 2026-06-04T23:09:26.698377 | INFO  | 6015780565   category_start                             | 6015780565 | category_start       | "                  |
| 104    | 2026-06-04T23:09:28.859209 | INFO  | 6015780565   category_selected   base:Броня             | 6015780565 | category_selected    | base:Броня         |
| 105    | 2026-06-04T23:09:35.699091 | INFO  | 6015780565   subcategory_selected   cat/cancel          | 6015780565 | subcategory_selected | cat/cancel         |

Изображение 24. Файл логирования

### 3.7.3. Мониторинг и диагностика

Для обеспечения стабильной работы реализованы:

- Мониторинг нагрузки - отслеживание количества запросов
- Анализ ошибок - классификация и приоритизация ошибок

### 3.8. Производительность и оптимизация

#### 3.8.1. Методы оптимизации производительности

Система использует несколько подходов для обеспечения высокой производительности (Таблица 14).

| Метод оптимизации        | Реализация                         | Эффект                         |
|--------------------------|------------------------------------|--------------------------------|
| Асинхронная обработка    | Использование <code>asyncio</code> | Масштабируемость, отзывчивость |
| Кэширование данных       | Локальное хранение CSV             | Снижение нагрузки на API       |
| Оптимизированные запросы | Целевое получение данных           | Уменьшение времени ответа      |
| Эффективные алгоритмы    | Оптимизированный поиск             | Быстрая обработка запросов     |

Таблица 14. Оптимизация производительности

#### 3.8.2. Масштабируемость

Архитектура системы позволяет легко масштабировать функциональность:

- **Добавление новых методов поиска**
- **Интеграция с другими платформами** - благодаря модульной структуре
- **Увеличение количества пользователей** - за счет асинхронной обработки

### 3.9. Заключение по главе

Разработанный Telegram-бот представляет собой законченное решение для доступа к информации об аукционных предметах Stalcraft. Система демонстрирует высокий уровень проработки архитектуры и реализации. Программа успешно сочетает в себе:

#### 1. Удобство использования:

- Интуитивно понятный интерфейс с минимальным количеством шагов для выполнения типовых задач
- Два взаимодополняющих метода поиска (по названию и по категориям) для различных сценариев использования

- Подробная карточка предмета с наглядным представлением статистики и лучших предложений
  - Многоуровневая навигация с возможностью отмены операций
2. Безопасность:
- Многоуровневая система контроля доступа с разделением прав
  - Криптографически стойкие одноразовые пароли с автоматическим удалением после использования
  - Ограничение частоты запросов для защиты от злоупотреблений
  - Изоляция административных функций с строгой проверкой прав
3. Надежность:
- Многоуровневая система обработки ошибок
  - Резервный механизм получения данных (парсинг) при недоступности API
  - Всеобъемлющее логирование событий в трех направлениях для диагностики
  - Автоматические уведомления администратора о критических событиях
4. Производительность:
- Асинхронная архитектура для обработки тысяч одновременных соединений
  - Многоуровневое кэширование для снижения нагрузки на базу данных
  - Эффективные алгоритмы пагинации и сортировки
5. Управляемость:
- Полнофункциональная административная панель с шестью основными функциями
  - Статистика использования для мониторинга нагрузки
  - Детальный журнал событий для аудита и анализа
  - Простота добавления нового функционала благодаря модульной архитектуре
6. Масштабируемость:
- Модульная структура с четким разделением ответственности
  - Абстракция доступа к данным через ORM для легкой миграции на другие СУБД
  - Stateless-архитектура для горизонтального масштабирования
  - Возможности функционального расширения без изменения существующего кода

Интеграция с системой парсинга данных (описанной в Главе 1) и подсистемой хранения на базе SQL (описанной в Главе 3) обеспечивает актуальность информации, целостность данных и эффективную работу

всего комплекса. Система готова к промышленному использованию и может быть легко расширена дополнительным функционалом.

## ГЛАВА 4. ХРАНЕНИЕ ДАННЫХ

### 4.1. Общая концепция

В разработанной системе для хранения информации применяется реляционная модель данных с использованием системы управления базами данных SQLite и объектно-реляционного отображения (ORM) SQLAlchemy. Выбор данной архитектуры обусловлен необходимостью обеспечения целостности данных, поддержки транзакций, возможности сложных запросов и перспектив масштабирования.

#### 4.1.1. Принципы организации хранения

Система хранения данных построена на следующих фундаментальных принципах:

##### Принцип 1: Разделение по назначению

Информация разделена на три независимые базы данных, каждая из которых отвечает за определенный аспект функционирования системы (Таблица 15)

| База данных | Назначение                             | Тип данных                     | Частота записи                         |
|-------------|----------------------------------------|--------------------------------|----------------------------------------|
| items.db    | Справочная информация о предметах игры | Статические/обновляемые данные | Периодически (при обновлении каталога) |
| users.db    | Данные авторизованных пользователей    | Динамические данные            | По мере регистрации/модификации        |
| logs.db     | Журнал событий и действий системы      | Потоковые данные               | Постоянно (каждое действие)            |

Таблица 15. Базы данных

Преимущества разделения:

- Изоляция сбоев - проблема в одной базе не влияет на работу остальных
- Гибкость резервного копирования - возможность независимого бэкапа критичных данных



- Оптимизация под нагрузку - разные настройки для разных типов операций
- Упрощение сопровождения - возможность обслуживания баз независимо

#### Принцип 2: Использование ORM

Объектно-реляционное отображение SQLAlchemy обеспечивает абстракцию между кодом приложения и структурой базы данных.

Преимущества ORM:

- Безопасность - автоматическое экранирование параметров защищает от SQL-инъекций
- Переносимость - возможность миграции на другую СУБД без изменения бизнес-логики
- Типизация - проверка типов данных на уровне Python
- Поддержка разработки - автодополнение в IDE, рефакторинг
- Выразительность - высокоуровневый API для сложных запросов

#### Принцип 3: Нормализация данных

Структура таблиц приведена к третьей нормальной форме (3NF) для устранения избыточности и аномалий.

Характеристики нормализации:

- Каждая таблица представляет одну сущность
- Все неключевые атрибуты зависят только от первичного ключа
- Отсутствуют транзитивные зависимости
- Связи между таблицами реализованы через внешние ключи

#### 4.1.2. Технологический стек

Система управления базами данных: SQLite

SQLite выбрана в качестве СУБД благодаря характеристикам, приведенным в (Таблица 16).

| Характеристика              | Описание                                                       | Преимущество для проекта                                       |
|-----------------------------|----------------------------------------------------------------|----------------------------------------------------------------|
| Встраиваемость              | Не требует отдельного серверного процесса                      | Простота развертывания, отсутствие дополнительных зависимостей |
| Компактность                | Полный движок занимает менее 500 КБ                            | Минимальные требования к ресурсам                              |
| Кроссплатформенность        | Работает на всех основных ОС                                   | Переносимость между средами разработки и эксплуатации          |
| ACID-транзакции             | Гарантия атомарности, согласованности, изоляции, долговечности | Надежность хранения данных                                     |
| SQL-совместимость           | Поддержка большинства стандартов SQL-92 и SQL-99               | Возможность использования стандартных запросов                 |
| Встроенная поддержка Python | Модуль sqlite3 в стандартной библиотеке                        | Отсутствие необходимости установки дополнительных драйверов    |

Таблица 16. Характеристики SQLite

### Объектно-реляционное отображение: SQLAlchemy 2.0

SQLAlchemy предоставляет мощный инструментарий для работы с реляционными базами данных на языке Python.

Компоненты SQLAlchemy в проекте:

1. Declarative Base - декларативное описание моделей данных через классы Python
2. Session - управление сессиями взаимодействия с базой данных
3. Query API - высокоуровневый интерфейс для формирования запросов
4. Connection Pooling - пул соединений для эффективного использования ресурсов
5. Event System - система событий для кастомизации поведения

## 4.2. Данные предметов (Item)

### 4.2.1. Назначение и структура

Таблица items (Изобр. 25) предназначена для хранения справочной информации о предметах игры Stalcraft. Эта информация используется для поиска предметов по названию и категориям, а также для отображения детальной информации при формировании карточки предмета.

|     | id        | name                 | main_cat... | sub_category    | created_at                 |
|-----|-----------|----------------------|-------------|-----------------|----------------------------|
|     | Filter... | Filter...            | Filter...   | Filter...       | Filter...                  |
| 159 | 6w0zp     | Экзоброня «Масть»    | Броня       | Боевой          | 2026-05-31 17:50:11.426298 |
| 160 | qjoz3     | Экзоброня «Медве...  | Броня       | Боевой          | 2026-05-31 17:50:11.426979 |
| 161 | g43k0     | Экзоброня «Мул»      | Броня       | Боевой          | 2026-05-31 17:50:11.427533 |
| 162 | 7lm56     | Экзоброня «Самсон»   | Броня       | Боевой          | 2026-05-31 17:50:11.428007 |
| 163 | zzjk2     | Экзоброня «Туз»      | Броня       | Боевой          | 2026-05-31 17:50:11.428464 |
| 164 | zzy9m     | Экзоскелет «Абори... | Броня       | Боевой          | 2026-05-31 17:50:11.428908 |
| 165 | 96npy     | «Полярник»           | Броня       | Комбинированная | 2026-05-31 17:50:11.429346 |
| 166 | 0r429     | Бандитский костюм    | Броня       | Комбинированная | 2026-05-31 17:50:11.429778 |
| 167 | m06my     | Бандитский костю...  | Броня       | Комбинированная | 2026-05-31 17:50:11.430244 |
| 168 | 1rkz2     | Бронекостюм «Арм...  | Броня       | Комбинированная | 2026-05-31 17:50:11.430698 |
| 169 | 4qlwo     | Бронекостюм «Бул...  | Броня       | Комбинированная | 2026-05-31 17:50:11.431121 |

Изображение 25. Таблица items

**Поле id:** Идентификаторы предметов в Stalcraft представляют собой короткие строки (обычно 4 символа), а не числовые значения. Использование строкового типа позволяет:

- Точно соответствовать формату, используемому в официальном API игры
- Сохранять совместимость при изменениях формата идентификаторов
- Использовать идентификаторы как непрозрачные строки без предположений о структуре

**Поле name:** Названия предметов на русском языке обычно не превышают 100 символов. Тип String без явного ограничения длины:

- Дает гибкость при добавлении новых предметов с длинными названиями
- Упрощает миграцию данных
- Достаточен для хранения любых названий предметов

**Поля `main_category` и `sub_category`:** Категориальная иерархия разделена на два поля вместо хранения полной строки вида «Оружие/Штурмовые винтовка». Такой подход позволяет:

- Выполнять запросы по любому уровню иерархии независимо
- Создавать составные индексы для комбинированной фильтрации
- Избегать дублирования данных (не хранить полную строку в каждом поле)
- Упростить навигацию по категориям в интерфейсе

**Поле `created_at`:** Временная метка автоматически заполняется при создании записи:

- Позволяет отслеживать возраст данных
- Упрощает выявление устаревших записей
- Обеспечивает аудит изменений каталога

## 4.3. Пользователи (User)

### 4.3.1. Назначение и структура

Таблица users предназначена для хранения информации об авторизованных пользователях системы, управления их доступом и аудита регистраций.

#### Поле user\_id как первичный ключ:

Используется идентификатор Telegram, а не автоинкрементный номер. Это обеспечивает:

- Естественную связь - прямое соответствие с внешним источником (Telegram API)
- Уникальность - гарантируется платформой Telegram
- Производительность - прямой поиск по ID без дополнительных проверок
- Простоту интеграции - не требуется преобразование идентификаторов

#### Поле username:

- Поиск конкретного пользователя для административных операций
- Формирование отчетов с группировкой по username
- Отображение списков пользователей с сортировкой

#### Опциональные поля first\_name и last\_name:

Поля могут быть NULL, так как не все пользователи указывают имя и фамилию в Telegram. Это позволяет:

- Корректно обрабатывать пользователей без указанных имен
- Не требовать обязательного заполнения полей
- Сохранять имеющуюся информацию при ее наличии

#### Поле auth\_date:

Дата авторизации хранится в текстовом формате ISO 8601:

- Упрощает логирование и отображение
- Не требует преобразования типов при выводе
- Обеспечивает сортировку в хронологическом порядке
- Совместим с функциями работы с датой в SQL

#### Поле is\_active для мягкого удаления:

Вместо физического удаления записи при блокировке пользователя устанавливается флаг is\_active=0. Это позволяет:

- Сохранять историю - история авторизаций и действий остается для аудита
- Восстанавливать доступ - возможность восстановления без потери данных
- Анализировать поведение - изучение активности заблокированных пользователей
- Поддерживать целостность - отсутствие разрывов во внешних ключах

## 4.4. Модель журнала событий (Log)

### 4.4.1. Назначение и структура

Таблица logs (Изобр. 21) предназначена для регистрации всех значимых событий в системе, обеспечивая наблюдаемость, диагностику проблем и аудит безопасности.

#### Поле **timestamp**:

Временная метка хранится в текстовом формате ISO 8601 (например, 2025-01-15T10:30:00). Такой выбор обусловлен рядом практических преимуществ:

- Лексикографическая сортировка - строки в формате ISO 8601 естественным образом упорядочиваются в хронологическом порядке при обычной алфавитной сортировке, что упрощает выборку событий по времени.
- Упрощение отображения - значение можно выводить пользователю без дополнительного преобразования типов, что снижает накладные расходы при формировании отчетов.
- Совместимость с SQL-функциями - большинство СУБД, включая SQLite, поддерживают работу с датами в текстовом формате через встроенные функции (DATE(), JULIANDAY() и др.).
- Удобство извлечения подстрок - дату или время можно получить простым срезом строки (например, первые 10 символов дают дату в формате YYYY-MM-DD), без необходимости вызова специальных функций парсинга.

#### Разделение **message** и **details**:

В структуре записи журнала текстовая информация разделена на два поля:

- **message** - стандартизированное сообщение в формате "user\_id | action | details", предназначенное для быстрого чтения человеком и вывода в консоль или файл.

- details - дополнительные детали события, сохраняемые отдельно для программного анализа и построения отчетов.

Такое разделение позволяет:

- быстро воспринимать основное содержание события при просмотре журнала;
- извлекать структурированные данные из поля details при необходимости автоматической обработки;
- поддерживать совместимость с различными системами анализа логов, каждая из которых может использовать свое поле в зависимости от задачи.

## 4.5. Менеджер баз данных DBManager

### 4.5.1. Архитектура и назначение

Класс DBManager представляет собой центральный компонент системы хранения данных, обеспечивающий управление множественными базами данных, сессиями и транзакциями.

#### Назначение менеджера:

1. **Централизация конфигурации** - все параметры подключения к базам данных определены в одном месте
2. **Управление множественными базами** - координация работы трех независимых баз данных
3. **Изоляция сессий** - обеспечение потокобезопасности через `scoped_session`
4. **Обработка ошибок** - централизованная логика `commit/rollback`
5. **Управление жизненным циклом** - корректное создание и закрытие соединений

## 4.6. Заключение по главе

В ходе разработки подсистемы хранения данных были приняты архитектурные решения, обеспечивающие баланс между простотой, производительностью, надежностью и возможностью масштабирования.

#### Основные достижения:

**1. Выбор реляционной модели:** Использование SQL и SQLAlchemy ORM обеспечивает структурированность данных, поддержку транзакций ACID,

возможность сложных запросов и легкую миграцию на другие СУБД без изменения бизнес-логики.

**2. Нормализованная схема данных:** Разделение на три таблицы (items, users, logs) с четкими связями и ограничениями целостности предотвращает аномалии, дублирование и обеспечивает согласованность данных.

**3. Абстракция управления соединениями:** Класс DBManager инкапсулирует работу с множественными базами данных, обеспечивает потокобезопасность через `scoped_session`, корректную обработку транзакций с автоматическим `rollback` при ошибках.

**4. Всеобъемлющее логирование:** Таблица logs с индексацией по времени, уровню и пользователю позволяет диагностировать проблемы, проводить аудит безопасности, анализировать использование системы и выявлять аномалии.

**5. Оптимизация через кэширование:** Использование `@lru_cache` для частых запросов снижает нагрузку на базу данных до 90% и обеспечивает мгновенный отклик для повторяющихся операций.

Реализованная система хранения данных соответствует текущим требованиям проекта, обеспечивает высокую производительность при типовых нагрузках и предоставляет четкие пути для роста и адаптации к изменяющимся условиям эксплуатации.



## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Официальная документация Stalcraft API [Электронный ресурс]. – Режим доступа: <https://api.stalcraft.net/> (дата обращения: 14.12.2025).
2. Документация aiogram (версия 3.x) [Электронный ресурс]. – Режим доступа: <https://docs.aiogram.dev/> (дата обращения: 14.12.2025).
3. Документация Telegram Bot API [Электронный ресурс]. – Режим доступа: <https://core.telegram.org/bots/api> (дата обращения: 14.12.2025).
4. Документация библиотеки Requests для Python [Электронный ресурс]. – Режим доступа: <https://docs.python-requests.org/> (дата обращения: 14.12.2025).
5. Документация BeautifulSoup [Электронный ресурс]. – Режим доступа: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/> (дата обращения: 14.12.2025).
6. Документация Selenium WebDriver [Электронный ресурс]. – Режим доступа: <https://www.selenium.dev/documentation/> (дата обращения: 14.12.2025).
7. Документация Pandas [Электронный ресурс]. – Режим доступа: <https://pandas.pydata.org/> (дата обращения: 14.12.2025).
8. Ramalho L. Fluent Python: Clear, Concise, and Effective Programming. – 2nd ed. – Sebastopol: O'Reilly Media, 2022. – 1016 p.
9. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. – Sebastopol: O'Reilly Media, 2017. – 614 p.
10. Официальная документация языка Python [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/> (дата обращения: 14.12.2025).
11. ChromeDriver – WebDriver for Chrome [Электронный ресурс]. – Режим доступа: <https://chromedriver.chromium.org/> (дата обращения: 14.12.2025).
12. RFC 4180: Common Format and MIME Type for Comma-Separated Values (CSV) Files [Электронный ресурс]. – Режим доступа: <https://www.rfc-editor.org/rfc/rfc4180> (дата обращения: 14.12.2025).
13. Стандарт ECMA-404: The JSON Data Interchange Syntax [Электронный ресурс]. – Режим доступа: <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/> (дата обращения: 14.12.2025).
14. Matyushkin L. Python и API: превосходное комбо для автоматизации работы с публичными данными // Proglib.io

- [Электронный ресурс]. – 2021. – 26 февраля. – Режим доступа: <https://proglib.io/p/python-i-api-prevoshodnoe-kombo-dlya-avtomatizacii-raboty-s-publicnymi-dannymi-2021-02-26> (дата обращения: 21.12.2025).
15. Real Python Tutorials. Async IO in Python: A Complete Walkthrough // Real Python [Электронный ресурс]. – Режим доступа: <https://realpython.com/async-io-python/> (дата обращения: 21.12.2025).
  16. Документация библиотеки SQLAlchemy [Электронный ресурс]. – Режим доступа: <https://docs.sqlalchemy.org/> (дата обращения: 14.12.2025).
  17. Документация SQLite [Электронный ресурс]. – Режим доступа: <https://www.sqlite.org/docs.html> (дата обращения: 14.12.2025).
  18. OAuth 2.0 Authorization Framework [Электронный ресурс]. – Режим доступа: <https://oauth.net/2/> (дата обращения: 14.12.2025).